electronics

MDPI

Article

Predictive Mamba-Enhanced Multi-Agent Reinforcement Learning Control for Virtual Coupling of High-Speed Trains

Han Hu [1], Qingsheng Feng [1], Zhun Han [1], Wangyang Liu [2] and Hong Li [3,*]

[1] School of Electrical Engineering, Dalian Jiaotong University, Dalian 116028, China; hhdjtu@163.com (H.H.); fqs@djtu.edu.cn (Q.F.); 13604228286@163.com (Z.H.)
[2] The Key Laboratory of Road and Traffic Engineering, Ministry of Education, Tongji University, Shanghai 201804, China; liuwy@tongji.edu.cn
[3] School of Railway Intelligent Engineering, Dalian Jiaotong University, Dalian 116028, China
* Correspondence: lihong@djtu.edu.cn

Abstract

Virtual coupling control of trains is a promising technology for improving railway capacity and operational efficiency. However, existing multi-agent reinforcement learning (MARL) approaches struggle to capture long-sequence temporal dependencies among train states in complex multi-train interaction scenarios, resulting in limited robustness and coordination stability. To address this issue, this paper proposes a Predictive Mamba-based Multi-Agent Soft Actor–Critic (PM-MASAC) framework. A Mamba-based state prediction module is embedded into the centralized Critic network to model historical state sequences and generate predictive state representations, thereby enhancing value estimation accuracy. In addition, a multi-agent aggregated prioritized experience replay (PER) mechanism is introduced to improve the utilization of critical cooperative samples and stabilize training. A hierarchical local–global reward structure is further designed to ensure individual tracking performance while promoting overall formation coordination. Experimental results under realistic railway operating conditions demonstrate that PM-MASAC achieves superior robustness compared with baseline MARL methods. Velocity and spacing tracking errors are maintained within 3% and 1%, respectively, and the steady-state formation success rate exceeds 95.7% in the training environment.

Keywords: virtual coupling; multi-agent soft actor-critic; predictive Mamba block; prioritized experience replay mechanism

Academic Editor: Krzysztof Zboiński

Received: 10 March 2026
Revised: 16 April 2026
Accepted: 21 April 2026
Published: 24 April 2026

Copyright: © 2026 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license.

1. Introduction

With the rapid growth of railway transportation demand, further improving transport efficiency and reducing energy consumption while ensuring operational safety has become a central research issue in modern railway systems. Traditional train operation control technologies are primarily based on the train schedule and movable block mechanism. Although these approaches satisfy existing transportation requirements to a certain extent, their operational efficiency is gradually approaching theoretical limits.

In recent years, the Vehicle-to-Vehicle Communication Based Train Control (VBTC) [1] system enables information sharing and cooperative decision-making among trains through vehicle-to-vehicle communication. Multiple trains can operate cooperatively at reduced headways without physical coupling and can dynamically couple and decouple

[2] according to passenger demand variations. This provides a new technical pathway for reducing headways and improving line capacity. Moreover, the reduced inter-train spacing introduces aerodynamic interactions, which can effectively decrease drag and improve energy efficiency. Schumann et al. [3] developed a railway operation simulation to study virtual coupling, demonstrating that it can effectively increase line capacity, with passenger throughput improved from 15,000 to 23,000 per hour per direction. Tian et al. [4] analyzed energy consumption in virtual coupling systems under different control modes, demonstrating a strong correlation between train operating intervals, control strategies, and energy efficiency. Fadhel et al. [5] investigated the aerodynamic benefits of virtual coupling using CFD (Computational Fluid Dynamics) simulations, showing that close train spacing can significantly reduce drag and improve energy efficiency. However, in dynamic and complex operational environments, achieving cooperative control and operational stability among multiple trains while ensuring safety remains a critical challenge for virtual coupling systems.

Existing research on virtual coupling control of trains predominantly adopts model-based control methods, such as proportional–integral–derivative (PID) control [6], linear quadratic regulator (LQR) [7], model predictive control (MPC) [8], and sliding mode control (SMC) [9]. Liu et al. [10] propose a tube-based model predictive control (MPC) approach and present simulation results that verify its stronger robustness and higher efficiency in maintaining asymptotic and string stability under disturbances and delays. He et al. [11] proposed an adaptive sliding mode controller for virtual coupling train control, enabling safe close-distance tracking while improving tracking performance and transportation efficiency. Luo et al. [12] employed a PID-based approach that integrates feedforward and PID controllers to achieve accurate tracking and stable formation control. Du et al. [13] developed a virtual coupling train group controller based on ILQR (Iterative Linear Quadratic Regulator), integrating a dynamic train model to achieve stable formation control. Felez et al. [14] presents a robust model predictive control scheme for virtual coupling in railways, which introduces two types of uncertainties to enhance operational safety while maintaining service efficiency. These studies have achieved progress in reducing headways and improving tracking accuracy. However, such approaches typically rely on fixed control rules or accurate dynamic models, making it difficult to characterize the complex and time-varying operating conditions in virtual coupling systems. As a result, the robustness and stability remain limited.

In recent years, Artificial Intelligence (AI) technologies [15] have provided new solutions for complex transportation system control. Reinforcement Learning (RL) [16], as an important branch of AI, addresses sequential decision-making problems under the Markov Decision Process (MDP) framework [17]. Due to its strong adaptability in nonlinear and strongly coupled systems, RL has been applied to train energy-saving control [18], autonomous driving [19], and traffic scheduling [20]. In virtual coupling scenarios, deep RL methods based on the actor–critic architecture can achieve cooperative control through interaction learning. Lang et al. [21] proposed a speed profile optimization method based on DQN (Deep Q-Network) for virtual coupling trains, which improves train synchronization and punctuality. Basile et al. [22] applied the DDPG (Deep Deterministic Policy Gradient) reinforcement learning algorithm to the virtual coupling control of automated train platoons, successfully achieving formation control. Chen et al. [23] combined the SAC algorithm with a traditional PID controller, effectively improving tracking accuracy and formation stability. Liao et al. [24] applied deep reinforcement learning (DRL) to train timetable rescheduling in metro systems, demonstrating strong learning efficiency and optimization capability. Mao et al. [25] designed a dual-layer deep reinforcement learning (DL-DRL) method for large-scale multi-UAV (Unmanned Aerial Vehicles) task schedul-

ing, significantly improving computational efficiency and generalization capability. Nevertheless, the above RL methods make decisions solely based on current observations. For the typical long-sequence, strongly coupled multi-agent cooperative control scenario of train virtual coupling, the variations in speed, position, and acceleration among trains exhibit significant temporal correlations. Decisions based only on current observations fail to capture long-term temporal dependencies and are sensitive to noise, resulting in unstable policies.

To address the above limitation, researchers have incorporated deep sequential models with temporal modeling capabilities, such as LSTM (Long Short-Term Memory) and Transformer, into RL frameworks to better exploit historical information and enhance future state prediction. Karimzadeh et al. [26] integrated an LSTM-based prediction framework with reinforcement learning to predict urban traffic trajectories and flow of passengers, improving prediction accuracy. Tseng et al. [27] proposed an LSTM-based predictive control framework combined with deep reinforcement learning, which accurately predicts future road density, vehicle speed, and flow of traffic, effectively reducing travel time and energy consumption. Liu et al. [28] utilized a Transformer-based state prediction model to enhance reinforcement learning policy training through self-supervised objectives, significantly improving sample efficiency in continuous control tasks. Liu et al. [29] proposed the Scene-Rep Transformer framework, which improves SAC-based autonomous driving decision-making in dense traffic scenarios by exploring latent information from sequence prediction. Although these methods have shown effectiveness in traffic prediction and autonomous driving, LSTM is limited by its recursive structure and suffers from gradient descent when modeling long sequence in multi-train cooperative decision-making tasks, while Transformer exhibits powerful global modeling capabilities, the lack of explicit state-space inductive bias can make it less suitable for modeling continuous dynamical systems.

Recently, the Mamba model, based on the Selective State Space Model (SSM), has emerged as a promising architecture for sequential modeling. With linear computational complexity and efficient long-sequence modeling capability, Mamba demonstrates significant advantages in temporal data processing. Several studies have incorporated Mamba into RL frameworks to enhance state representation and value estimation. Wang et al. [30] improved policy stability in the LocoMamba framework by performing hierarchical modeling of environmental states. Mustafa et al. [31] introduced the Mamba module into the value network to encode historical state sequences, enabling more efficient navigation. Ryu et al. [32] proposed the Mamba-DQN framework, which enhances the accuracy of Q-value estimation by selectively summarizing state sequences through the Mamba module. These findings indicate that Mamba can improve policy stability and convergence efficiency owing to its efficient long-sequence modeling ability.

Motivated by the above analysis, this paper focuses on the multi-train virtual coupling control problem and introduces a Mamba-based state prediction module into the centralized Critic network of the MASAC (Multi-Agent Soft Actor-Critic) framework. The historical state sequence is modeled to generate predictive state representations, thereby enhancing modeling capability. Furthermore, a multi-agent aggregated Prioritized Experience Replay (PER) mechanism and an automatic entropy adjustment strategy are incorporated during training to improve sample efficiency, exploration capability, and overall training stability. The main contributions of the paper are summarized as follows:

- A predictive Mamba-based multi-agent soft actor–critic framework (PM-MASAC) is proposed. By predicting the next-step state, the control performance of virtual coupled trains under dynamic operating conditions is significantly improved.
- A hierarchical local–global cooperative reward mechanism is constructed. While ensuring tracking performance, the mechanism guides agents to focus on the overall

operational state of the virtual formation, promoting cooperative behavior learning among trains.

- A multi-agent TD-error aggregated prioritized experience replay (PER) strategy is proposed. By strengthening the utilization of critical cooperative experiences, the training efficiency of multi-agent reinforcement learning is enhanced.
- Comprehensive validation under multiple typical train operating scenarios demonstrates that the proposed PM-MASAC framework outperforms baseline algorithms in cooperative control performance, training efficiency, and robustness, thereby verifying its effectiveness.

The rest of this paper is organized as follows. Section 2 formulates the multi-train virtual coupling problem as a Markov decision process (MDP) and defines the state space, action space, and reward function. Section 3 introduces the proposed PM-MASAC framework, including the Mamba-based state prediction mechanism, prioritized experience replay, and the update process of the policy and value networks. Section 4 presents comparative experiments and ablation studies under realistic railway operating scenarios. Section 5 concludes the paper and outlines future research directions.

2. Multi-Agent Virtual Train Coupling Decision Modeling

2.1. Problem Formulation

The objective of virtual coupling control is for each follower to adjust its position and speed to maintain the desired safe distance from the preceding train while synchronizing with the leader's velocity, ultimately achieving a stable, equidistant, and iso-velocity virtual coupling structure. The leader follows a predefined speed profile, serving as the reference for the formation. The followers acquire necessary state information through vehicle-to-vehicle communication and autonomously decide the acceleration to maintain a stable formation while satisfying safety constraints. The virtual coupling environment imposes physical constraints, including maximum speed, acceleration, and safety distance limits to ensure engineering feasibility. All trains operate along a single track in one direction, considering only longitudinal dynamics and neglecting lateral motion or turnout switching.

A simulation environment for multi-agent reinforcement learning (MARL) is constructed to evaluate the effectiveness of the proposed algorithm in virtual train coupling control tasks. The environment consists of three trains: a leader and two followers (Follower 1 and Follower 2), forming a typical longitudinal train formation scenario. Figure 1 illustrates the train control model based on the VBTC system.

However, the proposed framework is not limited to this configuration. The system is formulated in a decentralized multi-agent manner, where each following train makes decisions based only on local observations. Such a framework ensures that the proposed method can be extended to larger-scale coupling by increasing the number of agents, without requiring modifications to the policy structure or learning mechanism.

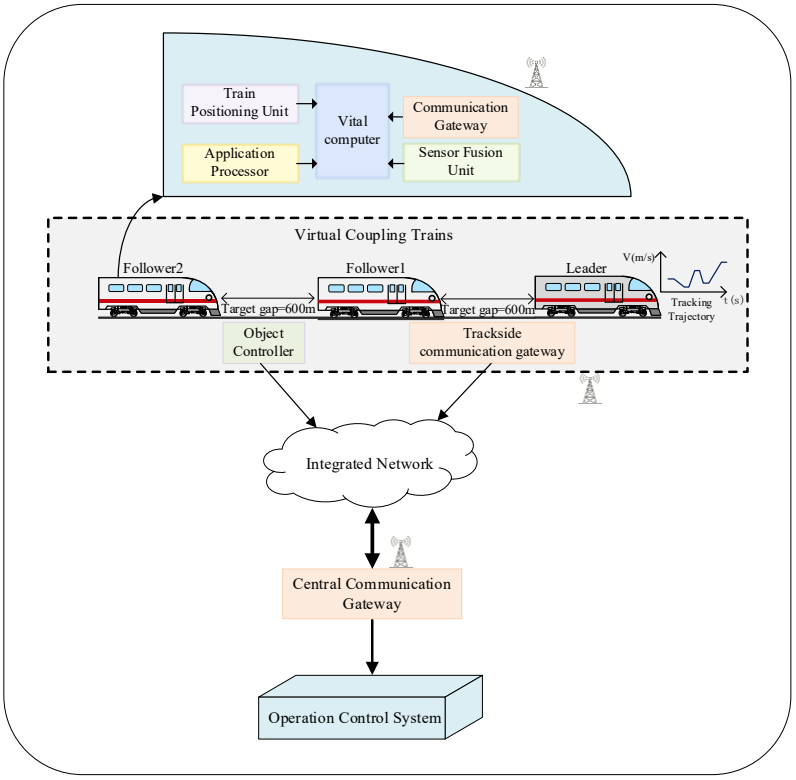


Figure 1. Framework of VBTC train control.

2.2. Markov Decision Process Modeling

2.2.1. Definition of State Space and Action Space

In the proposed framework, each follower is treated as an individual agent. At each decision step, the environment provides the agent with local observations and computes rewards based on the formation operation state to guide collaborative control strategy learning. The system is formulated as a continuous-action Markov Decision Process (MDP), represented by a quintuple (S, A, P, R, γ), where S is the state space, A is the action space, P denotes the state transition probability, R is the reward function, and γ is the discount factor for cumulative rewards.

For each agent i at time step t, the local observation vector is defined as $S_t = \left\{ \mathbf{s}_t \middle| \left[s_t^{i=1}, s_t^{i=2} \right] \right\}$, for each state $s_t^{i=1,2} = \left[x_t^i, v_t^i, a_t^i, x_t^{i-1}, v_t^{i-1}, a_t^{i-1}, \Delta d_t^i, \Delta v_t^i, v_t^0, a_t^0 \middle| s_t^i \in S_t \right]$, where x_t^i, v_t^i, a_t^i denote the position, speed, and acceleration of the agents; x_t^{i-1}, v_t^{i-1} a_t^{i-1} represent the preceding train's state; Δd_t^i and Δv_t^i are the relative distance and relative speed; and v_t^0, a_t^0 correspond to velocity and acceleration of the leader. Providing explicit relative information accelerates convergence while ensuring both local tracking safety and global coordination.

At each time step t, agent i generates a control action, defined as $A_t = \left\{ \mathbf{a}_t \middle| \left[a_t^{i=1}, a_t^{i=2} \right] \right\}$. By adaptively adjusting their accelerations according to the observed state, the follower trains regulate their relative velocity and inter-train distance with respect to both the preceding train and the leader, thereby achieving and maintaining the virtual coupling.

The discrete-time state update for agent i is expressed as:

$$\begin{cases} v_i\left(t+\Delta t\right)=v_i\left(t\right)+a_i\left(t\right)\cdot\Delta t \\ x_i(t+\Delta t)=x_i(t)+v_i(t+\Delta t)\Delta t \end{cases} \quad (1)$$

Benefiting from the data-driven, interactive learning paradigm of DRL, there is a reduced reliance on a precise mathematical model. In our study, the dynamic linear model described by Equation (1) generates coherent and plausible environmental interaction data during the training phase. It provides a sufficient basis for agents to learn the essential cooperative behaviors of speed tracking and spacing maintenance in virtual coupling.

2.2.2. Hierarchical Reward Function Design

Considering the "equidistant–iso-velocity–safety" characteristics of multi-train formation control, a hierarchical reward function integrating local rewards, global rewards, and penalties is designed to achieve multi-objective collaborative optimization:

$$R_i = R_i^{local} + R_{global} - P_i \quad (2)$$

1. Local Reward

The local reward is designed to guide the autonomous decision-making process of each follower train, encouraging stable and safe tracking behavior while maintaining smooth control inputs. The overall formulation is presented as follows:

$$R_i^{local} = w_1 \cdot r_i^{dist} + w_2 \cdot r_i^{vel} + w_3 \cdot r_i^{smooth} \quad (3)$$

where r_i^{dist}, r_i^{vel} and r_i^{smooth} denote the distance regulation reward, velocity coordination reward, and control smoothness reward, respectively.

(1) Distance Regulation Term

To ensure that the inter-train spacing converges toward the desired target distance, the normalized distance error is defined as:

$$e_i^d = \frac{\left|\Delta d_t^i - d_{target}\right|}{d_{target} + \varepsilon} \quad (4)$$

where Δd_t^i denotes the actual spacing between follower train i and its preceding train, d_{target} represents the desired inter-train distance and ε is a small constant introduced to prevent numerical instability.

The corresponding distance regulation reward is constructed as:

$$r_i^{dist} = \begin{cases} 1-e_i^d, & 0 \le e_i^d < 1, \\ 0, & e_i^d \ge 1. \end{cases} \quad (5)$$

This design ensures that the maximum reward is obtained when the actual spacing equals the target spacing. As the deviation increases, the reward decays linearly. When the deviation exceeds a specified bound, the reward is set to zero, thereby avoiding misleading or ineffective reward signals during policy learning.

(2) Velocity Coordination Term

The velocity coordination term captures the requirement of speed consistency in virtual coupling control. To simultaneously ensure local following stability and global velocity synchronization, global speed reference information is incorporated. This dual-level guidance mechanism promotes coordinated convergence of the virtual coupling system. The relative velocity error is calculated as:

$$e_i^{v_r} = \frac{\left| v_t^i - v_t^{i-1} \right|}{v_t^0 + \varepsilon} \tag{6}$$

where v_t^i denotes the velocity of agent i, v_t^{i-1} represents the velocity of the preceding train, and v_t^0 corresponds to the leader train velocity.

The global velocity error is calculated as:

$$e_i^{v_0} = \frac{\left| v_t^i - v_t^0 \right|}{v_t^0 + \varepsilon} \tag{7}$$

The velocity coordination reward is calculated through the weighted fusion of the two types of errors mentioned above:

$$r_i^{vel} = \lambda_1 \cdot (1.0 - e_i^{v_r}) + \lambda_2 \cdot (1.0 - e_i^{v_0}) \tag{8}$$

where $\lambda_1 = 0.6$ and $\lambda_2 = 0.4$ are weighting coefficients which are used to balance the contribution of relative velocity error and global velocity error. The reward is capable of guiding vehicles to realize the consistency of the overall speed while guaranteeing local tracking safety.

(3) Acceleration Smoothness Term

Ride comfort and equipment longevity are critical considerations in virtual coupling control. Drastic variations in acceleration may degrade passenger comfort and increase mechanical wear. Therefore, a jerk-based penalty is introduced to constrain abrupt control variations. The jerk of agent i is defined as:

$$jerk_i = \frac{\left| a_i^t - 2a_i^{t-1} + a_i^{t-2} \right|}{\Delta t}, \qquad t > 2 \tag{9}$$

where a_i^t denotes the current acceleration, a_i^{t-1} and a_i^{t-2} represent accelerations at previous time steps, and Δt is the time step. The jerk is calculated as the second-order finite difference of consecutive acceleration commands generated by the reinforcement learning policy, enabling more accurate comfort evaluation. The smoothness reward is formulated as:

$$r_i^{smooth} = 1 - clip(\frac{jerk_i}{\delta}, 0, 1) \tag{10}$$

where $\delta = 2$ is the fixed normalization coefficient, which ensures that the smoothness reward is on a comparable scale with other reward components, thereby maintaining a balanced multi-objective reward design.

(4) Stage-Based Weight Adjustment Mechanism

Virtual coupling control simultaneously involves safety, efficiency, and comfort objectives. To facilitate stable convergence under multi-objective optimization, a dynamic weight scheduling strategy based on training episodes is introduced. The weight coefficients of the distance regulation reward, velocity coordination reward, and comfort reward are adjusted across different training phases.

The specific parameter settings are listed in Table 1.

Table 1. Different settings of reward function weights.

Episode Range	Distance Weight	Velocity Weight	Comfort Weight
episode ≤ 350	0.5	0.3	0.2
350 < episode ≤ 600	0.3	0.4	0.3
episode > 600	0.2	0.3	0.5

In this study, the reward weights are designed based on a task-oriented strategy. Specifically, the weights are assigned to reflect different control priorities at different stages of training. The stage division is determined based on preliminary sensitivity analysis, which indicates that these transition points provide a good trade-off between convergence speed and control performance.

In the early stage, distance control is prioritized, with weights proportionally set as 0.5 to ensure safe spacing between trains while also considering velocity synchronization and action smoothness. In the middle stage, as training progresses, the proportion of velocity weight increases to 0.4, distance control slightly decreases to 0.3, achieving a balanced emphasis across control objectives. In the late stage, smoothness is prioritized, increasing comfort weight to 0.5, ensuring stable acceleration changes and stable virtual train coupling. By proportionally and dynamically adjusting the reward weights, agents can focus on different control objectives at different stages, ultimately achieving equally spaced, synchronized, and smooth virtual coupling.

2. Global Reward

To encourage agents to form a coordinated and consistent virtual coupling structure, a global reward mechanism is introduced. When all trains simultaneously satisfy the following conditions:

- the inter-train spacing error is within 5% of the target spacing;
- the velocity deviation is controlled within 2% of the target velocity.

The global coupling reward is triggered:

$$R_{global} = \begin{cases} 10.0, & e_i^d < 0.05, e_i^{v_0} < 0.02 \\ 0 \end{cases} \tag{11}$$

3. Safety Constrains

In virtual coupling control tasks, safety is the highest-priority objective. If only positive distance-based rewards are adopted, agents in the early training stage may obtain short-term rewards through aggressive acceleration or braking behaviors, potentially leading to rear-end collision risks or coupling instability. To address this issue, three additional penalty terms are incorporated into the reward function to constrain and suppress potentially hazardous behaviors:

$$P_i = P_i^{danger} + P_i^{collision} + P_i^{loose} \tag{12}$$

(1) Dangerous Proximity Penalty

This penalty characterizes situations in which the train operates within a potential safety risk zone. It guides the agent to decelerate or adjust its behavior in order to restore a safe spacing margin. When the inter-train distance falls below the minimum safe gap d_{min}, the penalty is defined as:

$$P_i^{danger} = 2 \cdot clip\left(\frac{|s_i|}{d_{min}}, \quad 0, \quad 1 \right) \qquad s_i < 0 \tag{13}$$

where $s_i = \Delta d_t^i - d_{min}$ denotes the spacing margin.

(2) Collision Penalty

This term strictly constrains unacceptable safety-critical events and significantly reduces the probability of rear-end collisions during training. Once a collision occurs, a strong negative penalty is immediately imposed:

$$P_i^{collision} = 5 \qquad \Delta d_t^i < 0 \tag{14}$$

(3) Excessive Spacing Penalty

To prevent the inter-train distance from becoming excessively large and causing structural looseness of the virtual coupling formation, an additional penalty is introduced. When the actual spacing exceeds twice the target spacing, the excessive spacing penalty is activated:

$$p_i^{loose} = clip\left(\frac{\Delta d_t^i}{d_{target}} - 2,\ 0,\ 1\right) \qquad \Delta d_t^i > 2d_{target} \tag{15}$$

It is worth noting that safe-related penalties are assigned higher priority than the jerk term above. In safe-critical situations, the agent is allowed to generate large control variations to ensure collision avoidance.

3. Proposed PM-MASAC Framework

3.1. PM-MASAC Architecture

The proposed Predictive Mamba-based Multi-Agent Soft Actor–Critic (PM-MASAC) adopts a multi-agent reinforcement learning framework based on Centralized Training with Decentralized Execution (CTDE). The overall structure of the proposed PM-MASAC framework is illustrated in Figure 2. The actor and critic networks are implemented as Multilayer Perceptron (MLP) with three hidden layers containing 256, 256, and 128 neurons, respectively.

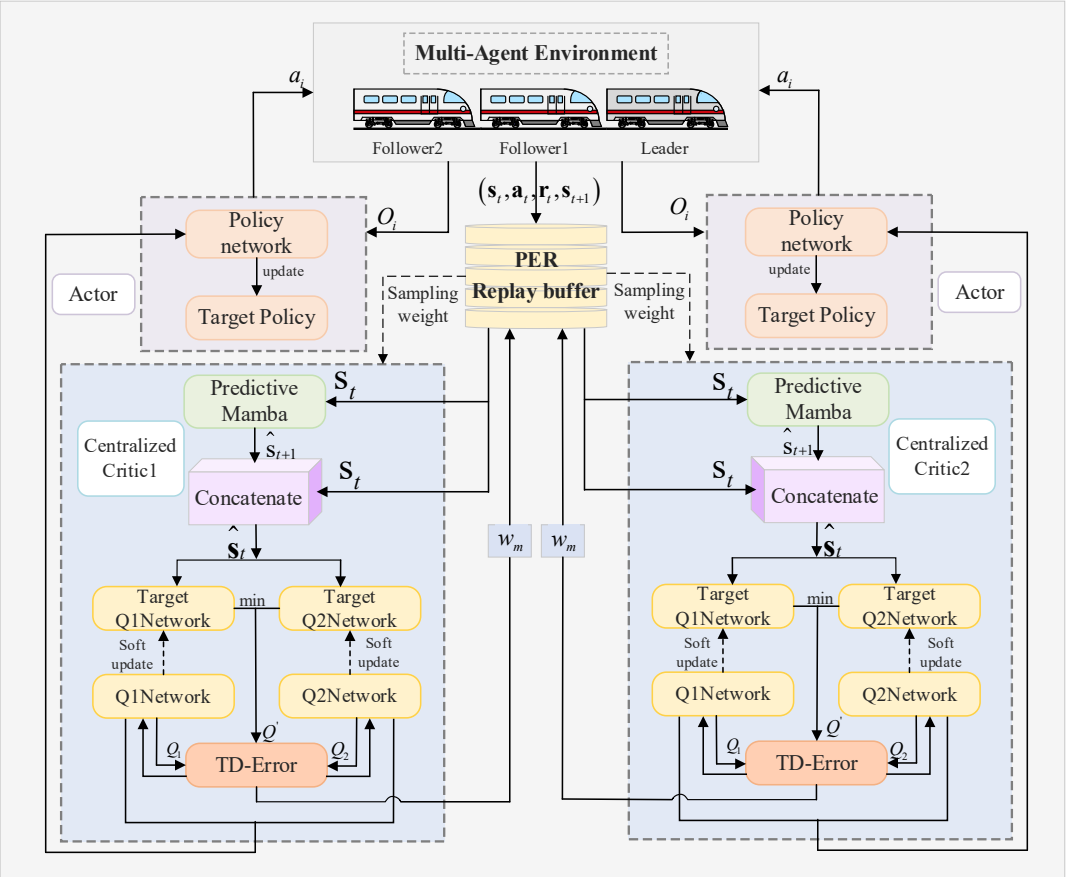


Figure 2. Network Architecture of PM-MASAC.

In Figure 2, O_i denotes the local observation of agent *i*, where each agent i generates an action a_i through its policy network. The VBTC receives the joint action vector and updates the system dynamics of all trains accordingly.

In the proposed PM-MASAC framework, each train is treated as an independent agent. The training follows the centralized training and decentralized execution (CTDE) paradigm. Specifically, each agent maintains an independent actor network, meaning that policy parameters are not shared across agents. This design allows each agent to learn its own control policy based on local observations. Additionally, the entropy-regularized policy optimization mechanism introduced in MASAC ensures sufficient exploration, thereby improving policy robustness.

For the critic, a centralized structure is adopted, where each agent is equipped with its own critic network that takes the global state and joint actions of all agents as input. Therefore, while the critic utilizes centralized information during training, the critic networks are not shared among agents.

To capture the temporal evolution characteristics of train operation state, a Mamba-based state prediction module is incorporated into the framework. This module predicts future states $\hat{\mathbf{s}}_{t+1}$ based on the current observation sequence. The predicted states are concatenated with the current states to form an enhanced state representation $\hat{\mathbf{s}}_t$, which is then fed into the critic network for value evaluation.

Moreover, to improve sample efficiency during training, a prioritized experience replay (PER) mechanism [33] is integrated into the algorithm. Unlike conventional uniform sampling, PER assigns different sampling probabilities to experiences based on their temporal-difference errors, allowing experiences with higher learning value to be sampled more frequently, thereby accelerating policy convergence and improving training stability.

Based on the above framework, each agent's actor network is updated by maximizing the expected entropy-regularized return. The entropy regularization term penalizes overly deterministic policies, effectively preventing the policy from prematurely converging to suboptimal solutions. The corresponding actor objective function is given by:

$$J(\pi_i) = E_{o_i \sim D, a_i \sim \pi_i}\left[\alpha \log \pi_i\left(a_t^i \mid o_i\right) - Q_\phi\left(\hat{\mathbf{s}}_t, a_t^i\right)\right] \tag{16}$$

where $\hat{\mathbf{s}}_t$ denotes the enhanced state, a_t^i represents the joint action, D is the experience replay buffer, and α is the entropy coefficient which is used to adjust the weight of exploration intensity.

A fixed entropy coefficient may lack adaptability across different training phases of virtual coupling control. The entropy coefficient is dynamically updated by minimizing the entropy loss function, driving it toward a predefined target entropy and ensuring adequate policy stochasticity. The update formula is given by:

$$L(\alpha) = E_{a_i \sim \pi_i}\left[-\alpha\left(\log \pi_i\left(a_t^i \mid o_i\right) + \mathrm{H}_{target}\right)\right] \tag{17}$$

where H_{target} is the target entropy, which guides the policy toward a desired level of stochasticity.

The parameters of the actor and critic networks are updated using stochastic gradient descent according to the aforementioned objectives, and the target networks are updated via soft updates to ensure training stability. The equation is as follows:

$$\phi_j^{'} \leftarrow \tau\phi_j + (1-\tau)\phi_j^{'} \tag{18}$$

where τ is the soft update coefficient.

3.2. Design of the Mamba Prediction Module

Deep reinforcement learning estimates value functions and learns control policies through interaction with the environment. The Q-network evaluates the value of state–action pairs, and its estimation accuracy directly affects policy optimization performance

and control stability. However, in virtual coupling scenarios, key system variables—including velocity, inter-train spacing, and operational constraints—exhibit pronounced temporal correlations. Conventional multilayer perceptron (MLP)-based Q-networks are insufficient for capturing deep feature interactions and temporal dependencies embedded in state–action pairs. In particular, they neglect the crucial reference significance of the next-state information. In dynamic and complex environments, this limitation may reduce policy generalization and robustness.

To address this issue, a Mamba-based sequence modeling architecture is introduced as a state predictor at the input stage of the Q-network.

The proposed Mamba structure is illustrated in Figure 3.

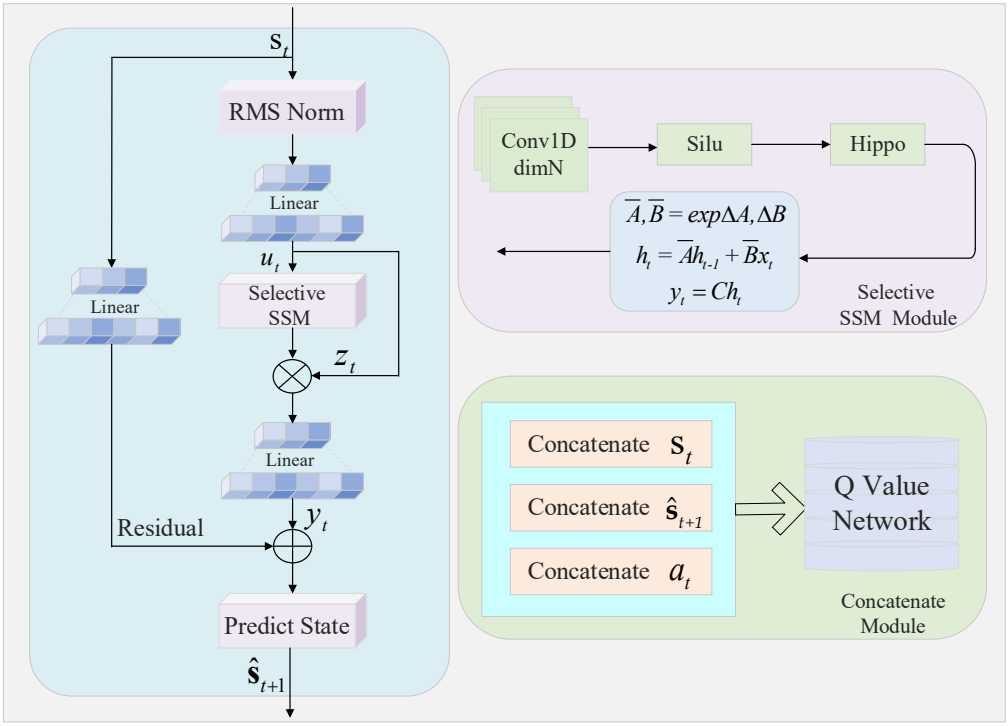


Figure 3. Framework of Mamba.

Within this framework, the Mamba module receives the current state s_t as input which is first normalized using an RMSNorm (Root Mean Square Layer Nromalization) layer to mitigate numerical instability caused by heterogeneous physical scales. The normalized features are then projected into a higher-dimensional latent space through a linear transformation and decomposed into a main branch u and a gating branch z, enabling dynamic representation learning and selective information control. The selective mechanism in the Mamba blocks is directly driven by the physical state variables of the train system. Within the selective state space module, a depthwise one-dimensional convolution (Conv1D) is employed to extract local temporal features. The SiLU (Sigmoid Linear Unit) activation function is further utilized to ensure smooth nonlinear transformations, thereby preserving the continuity of system dynamics. The state transition matrix A is initialized using the HiPPO (High-Order Polynomial Projection Operators) formulation, providing a multi-timescale memory mechanism. The predicted state is concatenated with the current state to construct an enhanced state representation $\hat{s}_t = [s_t \,||\, \hat{s}_{t+1}]$, which is then fed into the Q-network. This enhanced state, together with the current action a_t^i, forms an extended state–action pair for value function estimation.

The state prediction process above is primarily grounded in the State Space Model (SSM), which characterizes the temporal evolution of dynamic systems. The concept of SSM originates from continuous linear time-invariant (LTI) systems, typically described by linear ordinary differential equations (ODEs). Through the evolution of latent hidden states $h(t) \in \mathbb{R}^N$, SSM maps input sequences to output sequences. The state equation is expressed as:

$$\begin{cases} h'(t) = \mathrm{A}h(t) + \mathrm{B}x(t) \\ y(t) = \mathrm{C}h(t) \end{cases} \tag{19}$$

where $\mathrm{A} \in \mathbb{R}^{N \times N}$ is the state transition matrix, and $\mathrm{B} \in \mathbb{R}^{N \times 1}$ and $\mathrm{C} \in \mathbb{R}^{N \times 1}$ are projection matrices that map the input to the state space and the state to the output space, respectively.

To embed the continuous-time SSM into discrete sequence-based deep learning models, discretization is required. The Zero-Order Hold (ZOH) method is commonly adopted, introducing a time-step parameter Δ. The continuous parameters A and B are transformed into discrete parameters $\overline{\mathrm{A}}$ and $\overline{\mathrm{B}}$:

$$\begin{cases} \overline{\mathrm{A}} = \exp(\Delta \mathrm{A}) \\ \overline{\mathrm{B}} = (\Delta \mathrm{A})^{-1} (\exp(\Delta \mathrm{A}) - \mathrm{I}) \cdot \Delta \mathrm{B} \end{cases} \tag{20}$$

when the time-step is sufficiently small, $\overline{\mathrm{B}} \approx \Delta \mathrm{B}$. After discretization, the recursive update equations with discrete step k become:

$$\begin{cases} h[k] = \overline{\mathrm{A}}h[k-1] + \overline{\mathrm{B}}x[k] \\ y[k] = Ch[k] \end{cases} \tag{21}$$

The discrete SSM can be represented from a convolutional perspective. By unrolling the recurrence relation, the system output can be expressed as the convolution between the input sequence and a convolution kernel $\overline{\mathrm{K}}$:

$$y = x \cdot \overline{\mathrm{K}} = x \cdot \left(\mathrm{C}\overline{\mathrm{B}}, \mathrm{C}\overline{\mathrm{AB}}, \cdots, \mathrm{C}\overline{\mathrm{A}}^{L-1}\overline{\mathrm{B}} \right) \tag{22}$$

where *L* denotes sequence length. This convolution enables efficient parallel computation via Fast Fourier Transform (FFT), forming the basis of linear state space layers.

Traditional SSM-based neural architectures treat matrices A and B as directly learnable parameters. However, in complex sequence modeling tasks, such parameterization often limits long-range dependency modeling. To address the issue, Mamba adopts the Hippo-based initialization strategy proposed in the S4 model [34]. The Hippo matrix provides an optimal solution for online function approximation, enabling superior long-range dependency representation. Its initialization form is given by:

$$\begin{cases} \mathrm{A}_{nk} = -\begin{cases} \sqrt{(2n+1)(2k+1)} & , if\ n > k \\ n+1 & , if\ n = k \\ 0 & , if\ n < k \end{cases} \\ \mathrm{B}_n = \sqrt{2n+1} \end{cases} \tag{23}$$

Although Hippo initialization significantly improves long-term modeling ability, conventional SSMs remain constrained by the linear time-invariant model assumption. Their parameters do not dynamically update, which limits their adaptive capability to input. Therefore, the Mamba architecture introduces a selective mechanism [35]. In this architecture, key parameters—including $\mathrm{B} \in \mathbb{R}^{B \times L \times N}$, $\mathrm{C} \in \mathbb{R}^{B \times L \times N}$ and $\Delta \in \mathbb{R}^{B \times L \times D}$ are no

longer fixed. Instead, they are dynamically generated from the input sequence $x \in \mathbb{R}^{B\times L\times D}$ through linear projections:

$$\mathrm{B}(x), \mathrm{C}(x), \Delta(x) = \mathrm{LinearProjection}(x) \tag{24}$$

This selective mechanism enables the Mamba model to adaptively focus on informative temporal features while suppressing redundant or less relevant historical information based on the input sequence. In the virtual coupling control task, state variables such as velocity and position exhibit strong temporal continuity under normal operation, while also experiencing local abrupt variations induced by control actions and dynamic interaction among trains.

This capability provides an expressive representation of the temporal structure of multi-train coordination processes, which serves as the main motivation for integrating the Mamba module into the Q-network.

3.3. Design of the Multi-Agent Prioritized Experience Replay Mechanism

In reinforcement learning, experience replay improves sample utilization efficiency and mitigates temporal correlations between samples by storing and randomly sampling interaction experiences generated during the agent–environment interaction process. However, in multi-train virtual coupling control scenarios, different state transition samples contain significantly different levels of cooperative control information. Traditional uniform sampling strategies cannot distinguish the relative importance of samples and therefore fail to fully exploit critical interaction data. Therefore, under the centralized training with decentralized execution (CTDE) framework, this study proposes a hierarchical reward-driven prioritized experience replay (PER) mechanism to improve sample efficiency and training stability in multi-agent cooperative control tasks. The proposed mechanism guides the training process to focus more on high-value interaction samples.

In the replay buffer, each interaction sample is defined as:

$$e_i = \left(o_i, a_i^1, \cdots, a_i^N, r_i^1 \cdots, r_i^N, o_i^{'}, d_i\right) \tag{25}$$

where o_i and $o_i^{'}$ denote the joint observations at time steps t and t + 1, respectively; a_i^N and r_i^N represent the actions and rewards of agent N; d_i indicates the termination flag; and N denotes the number of agents. The capacity of the replay buffer is set to 5×10^4.

To reflect the importance of different experiences for policy updates, a proportional priority sampling strategy is adopted, where the sampling probability of the i-th experience is defined as:

$$P\left(i\right) = \frac{P_i^\alpha}{\sum_j P_j^\alpha} \tag{26}$$

where P_i denotes the priority of the i-th experience and $\alpha \in \left[0,1\right]$ is the priority weight factor. When $\alpha = 0$, the sampling degenerates into uniform sampling.

Since prioritized sampling changes the original distribution of experience samples, directly using these sampled data for gradient updates may introduce estimation bias. To address this issue, importance sampling (IS) weights are introduced to correct the loss function:

$$w_i = \left(\frac{1}{B}\cdot\frac{1}{P(i)}\right)^\beta \tag{27}$$

where $\beta \in \left[0,1\right]$ is the bias compensation factor and B denotes the replay buffer capacity.

Within the CTDE framework, the centralized critic evaluates the action value based on the joint state–action pair. For agent i, the target Q-value [36] is defined as:

$$y_i = r_i + \gamma\left[\min_{j=1,2} Q_j'\left(\hat{\mathbf{s}}_t, a_{t+1}^i\right) - \alpha \log \pi\left(a_{t+1}^i, o_i^{t+1}\right)\right] \tag{28}$$

where r_i represents the immediate reward obtained by agent i, which consists of both local rewards and global rewards; γ is the discount factor; a_i^{t+1} denotes the joint action sampled from the current policies at the next time step; and α is the entropy coefficient. $\hat{\mathbf{s}}_t$ represents the enhanced state generated by concatenating the current state with the Mamba-predicted next state.

The temporal-difference (TD) error for agent i at time step t is defined as:

$$\delta_t^i = \left(Q_1\left(\hat{\mathbf{s}}_t, a_t^i\right), Q_2\left(\hat{\mathbf{s}}_t, a_t^i\right)\right) - y_i \tag{29}$$

In traditional PER, the priority is updated using the TD error of a single agent. However, in multi-agent scenarios, such a strategy may introduce instability due to noise from individual agents. To address this issue, a multi-agent TD-error aggregation strategy is proposed. For each experience sample, TD errors from all agents within a training cycle are aggregated and averaged:

$$\overline{\delta}_t = \frac{1}{N}\sum_{i=1}^{N}\left|\delta_t^i\right| \tag{30}$$

In terms of the priority update mechanism, experience priorities are typically updated immediately after each network update based on the latest TD error. However, in multi-train virtual coupling scenarios, the system often experiences strong fluctuations during non-steady-state phases, such as startup, acceleration, braking, and coupling adjustment. During these phases, TD errors may vary significantly within a short time interval. Frequent priority updates based on instantaneous errors may amplify stochastic disturbances and lead to unstable sampling distributions. To mitigate this issue, a delayed priority update mechanism is introduced. For each experience sample, the aggregated multi-agent TD error is accumulated over K consecutive replay update steps, and the priority is updated only after reaching the predefined update interval:

$$p_i \leftarrow \frac{1}{K}\sum_{u=1}^{K}\overline{\delta}_i^{(u)} + \varepsilon \tag{31}$$

where $\overline{\delta}_i^{(u)}$ denotes the aggregated TD error of the i-th experience in the u-th update.

During parameter updates, the IS weights defined in Equation (27) are applied to weight the squared error loss of the critic network. The prediction module is trained implicitly via the critic objective, rather than using an explicit supervised MSE loss.

The loss function is defined as:

$$L\left(\phi_j\right) = E_{\left(\hat{\mathbf{s}}_t, a_t, r_t, s_{t+1}\right)\sim D}\left[w_i\left(Q_{\phi_j}(\hat{\mathbf{s}}_t, a_t^i) - y_i\right)^2\right] \tag{32}$$

The update procedures enable the MASAC framework to fully leverage Mamba-enhanced state information, PER sampling weights, and entropy regularization, thereby achieving stable and efficient policy learning for multi-agent train formation control.

3.4. Algorithm Pseudocode

Algorithm 1 presents the training procedure of the PM-MASAC algorithm for the virtual coupling control task. Under the centralized training with decentralized execution (CTDE) framework, agents continuously interact with the environment to generate state transition samples, which are stored in a prioritized experience replay buffer. When the

number of stored samples satisfies the training requirements, batch sampling is performed according to the priority distribution, and the Critic network, Actor network, and temperature parameter are updated. Meanwhile, to mitigate the influence of short-term fluctuations in TD errors on the stability of the sampling distribution, a delayed priority update strategy is adopted. This mechanism effectively improves training stability and sample utilization efficiency.

Algorithm 1. PM-MASAC with PER for virtual train formation control

The PM-MASAC

Input:

Environment $\boldsymbol{E}$ of the three trains formation scenario;

Actor networks π_{θ_i};

Mamba-*Q* Critic networks $Q_{\phi 1}$, $Q_{\phi 2}$ with target networks $Q_{\phi 1}^{'}$, $Q_{\phi 2}^{'}$;

Hyperparameters initialization: n_{agents}, γ, τ, α, lr;

PER replay buffer B with initialization parameters α_{PER} and β_{PER};

Output:

Trained policy π_{θ_i}.

Procedure:

Interaction Process

1. **Initialization:**
   Initialize actors π_{θ_i};
   Initialize predictive mamba centralized critics $Q_{\phi 1}, Q_{\phi 2}$, and target critics $Q_{\phi 1}^{'}, Q_{\phi 2}^{'}$;
   Initialize log entropy log $\alpha \leftarrow 0$;
   Initialize PER replay buffer B as empty;
   Initialize step counter $t \leftarrow 0$;
2. **for** each episode **do**
3. Reset environment $\boldsymbol{E}$;
4. Obtain initial state $S_t \leftarrow S_{t=0}$;
5. **for** each step **do**
6. **for** each agent i **do**
7. Sample actions $\hat{a}_t^i \sim \pi_{\theta_i}\left(\cdot \mid s_t^i\right)$ from actor networks;
8. **end for**
9. Execute joint action $\mathbf{a}_t$ in environment $\boldsymbol{E}$;
10. Observe next state s_{t+1}, reward r_t, and **done**;
11. Store transition $\left(\mathbf{s}_t, \mathbf{a}_t, \mathbf{r}_t, \mathbf{s}_{t+1}, \mathbf{done}\right)$ into B with maximum priority;
12. $S_t \leftarrow S_{t+1}$;

Update Process

13. **if** size(B) ≥ batch size, **then:**
14. Sample batch from B using priority distribution $P(i)$ based on (26);
15. Compute importance-sampling weights W_i based on (27);
16. **for** each agent i **do**
17. Update Critic networks based on (32);
18. Update Actor network based on (16);
19. Update temperature parameter α based on (17);
20. **end for**
21. Soft-update target parameters $\phi_j^{'}$ based on (18);
22. Accumulate TD-errors for sampled transitions;
23. **if** t mod K = 0,**then**:

24. Update priorities in replay buffer B;
25. **if** done **break**;
26. **end for**
27. **end for**
28. **Return** policy π_{θ_i} .

4. Experimental Simulation and Analysis

To comprehensively evaluate the effectiveness of the proposed method, this study first determines the optimal combination of key algorithm parameters through parameter sensitivity analysis. Subsequently, comparative experiments and ablation studies are conducted to verify the respective contributions of the MASAC framework, the Mamba prediction module, and the prioritized experience replay (PER) mechanism, as well as their combined synergistic effects. Finally, the overall performance and generalization capability of the proposed algorithm are systematically evaluated using both the training dataset and an independent test set, ensuring that the learned policy can be stably applied to multiple operating conditions after training.

4.1. Experimental Setup

The experiments are conducted in a self-defined multi-agent simulation environment, TrainFormationEnv, developed under the PettingZoo (Farama Foundation, New York, NY, USA) framework. All experiments are implemented using Python 3.9 (Python Software Foundation, Wilmington, DE, USA) and executed on the same hardware and software platform to ensure fairness and consistency in algorithm comparisons.

During the experiments, the leading train follows a predefined speed profile, while the following trains learn acceleration control strategies autonomously through the proposed PM-MASAC algorithm to track the target velocity and target spacing, thereby achieving a virtual coupling state. To ensure reproducibility, all algorithms adopt identical environment configurations and hyperparameter settings during training and evaluation, and a fixed random seed is used throughout the experiments. The training parameters are summarized in Table 2.

Table 2. Training parameters.

Meaning	Parameter	Value
Discount factor	γ	0.99
Learning rate	lr_Q, lr_π, lr_α	0.0001, 0.001, 0.001
Size of replay buffer	D	50,000
Size of sampling batches	B	256
Soft update parameter	τ	0.01
Number of sampling episodes	N_s	800
Maximum training steps per episode	T_e	2000
Entropy Temperature Coefficient	α	0.0001
The priority Coefficient of PER	α_p	0.6
Balance factor	β	0.4
Time step	Δt	0.3
Random seed	seed	58
Prediction horizon	P_h	1
minimum log std	$log\sigma_{min}$	−20
maximum log std	$log\sigma_{max}$	2

To quantitatively compare the control performance of different algorithms, several evaluation metrics are adopted. Except for the virtual coupling success rate, all metrics are calculated separately for the two following trains and then averaged to obtain the overall formation performance. The steady-state condition is defined as the two following trains simultaneously satisfying the spacing error within ±5% and the velocity error within ±2%.

1. Setting Time: The time when the follower train remains in the steady-state for five consecutive time steps.
2. Spacing Error: The absolute value of the relative error between the actual inter-train distance and the desired spacing.
3. Velocity Error: The absolute value of the relative error between the follower and preceding train.
4. MSE_v: Mean squared error of speed deviation.
5. MSE_a: Mean squared error of acceleration deviation, reflecting ride comfort.
6. Virtual Coupling Success Rate: The ratio of time steps where both followers maintain steady state simultaneously.

4.2. Sensitivity Analysis

4.2.1. Sensitivity Analysis of Hyperparameter

To investigate the impact of key hyperparameters on algorithm performance and training stability, a sensitivity analysis is conducted on the learning rate (lr), soft update coefficient, discount factor, PER priority weight coefficient and reward stage division. The cumulative reward is used as the evaluation metric, and the experimental results are presented in Figure 4.

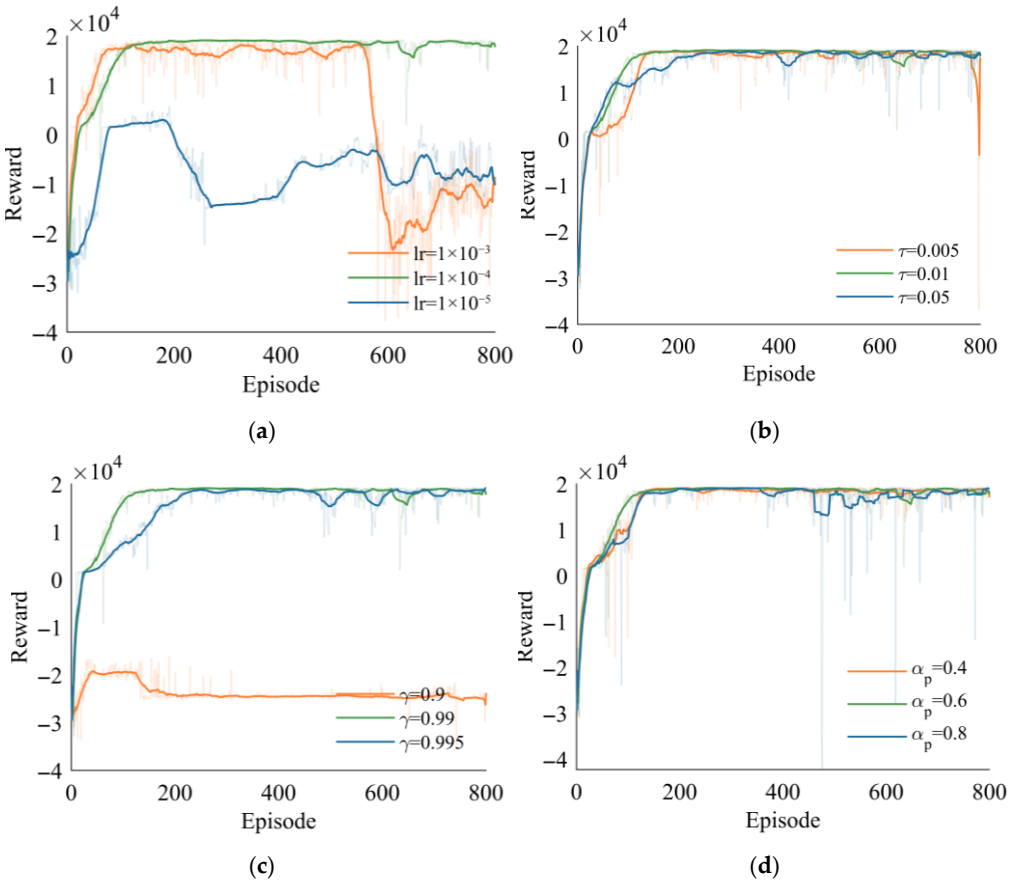


Figure 4. Sensitivity analysis of key hyperparameters in PM-MASAC: (**a**) the learning rate lr; (**b**) soft update coefficient τ; (**c**) discount factor γ; (**d**) PER priority weight α_p.

https://doi.org/10.3390/electronics15091823

The results of the hyperparameter sensitivity analysis indicate that the proposed algorithm exhibits differentiated sensitivity to different types of hyperparameters. Among them, the learning rate and discount factor have a more significant impact on training stability and convergence speed.

When the learning rate is set too high, the policy tends to oscillate in the later stages of training; conversely, while an excessively small learning rate may prevent the agent from learning an optimal policy. For the discount factor, a higher value delays policy convergence, while a lower value causes the agent to focus more on short-term rewards, limiting improvements in cumulative rewards.

In contrast, variations in the soft update coefficient and the PER priority weight coefficient within reasonable ranges have relatively limited influence on the final performance.

Based on the above analysis, the hyperparameters are finally set as learning rate lr = 1×10^{-4}, soft update coefficient τ = 0.01, discount factor γ = 0.99, and PER priority weight coefficient α_p = 0.6,which provide a reliable parameter foundation for the subsequent experiments.

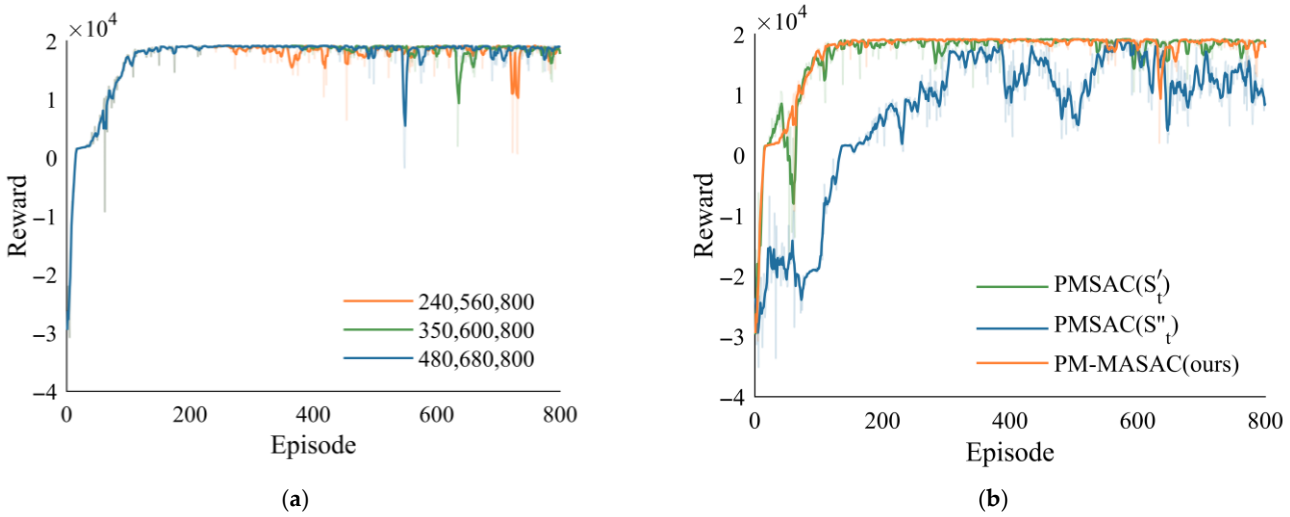
4.2.2. Sensitivity Analysis of MDP Components

To comprehensively evaluate the impact of design choices on the learning performance, a series of sensitivity and ablation experiments are conducted to investigate how different reward stage partition strategies and state variable combinations affect training stability and cumulative rewards. This analysis aims to provide insights into the effectiveness and robustness of the proposed design, as well as to justify the selected reward stage division and state representation used in subsequent experiments.

The ablation settings of state variables are summarized in Table 3 and the experimental results are presented in Figure 5.

Table 3. Ablation settings of state variables.

Algorithm	State Space
PM-MASAC S_t'	$s_t^{i=1,2} = \left[x_t^i, v_t^i, x_t^{i-1}, v_t^{i-1}, \Delta d_t^i, \Delta v_t^i, v_t^0 \middle\| s_t^i \in S_t \right]$
PM-MASAC S_t''	$s_t^{i=1,2} = \left[x_t^i, v_t^i, a_t^i, x_t^{i-1}, v_t^{i-1}, a_t^{i-1}, \Delta v_t^i, v_t^0, a_t^0 \middle\| s_t^i \in S_t \right]$
PM-MASAC (Ours)	$s_t^{i=1,2} = \left[x_t^i, v_t^i, a_t^i, x_t^{i-1}, v_t^{i-1}, a_t^{i-1}, \Delta d_t^i, \Delta v_t^i, v_t^0, a_t^0 \middle\| s_t^i \in S_t \right]$



(**a**) (**b**)

Figure 5. Sensitivity analysis of MDP components: (**a**) reward stage division; (**b**) state variables.

Figure 5a shows that different stage partition strategies lead to only minor variations in performance in terms of final convergence level. Among the three configurations, the green curve (350, 600, 800) achieves the most balanced behavior. In contrast, the orange curve (240, 560, 800) and the blue one (480, 680, 800) show relatively noticeable fluctuations during training. Considering both training stability and performance balance, the training process is divided into three stages with boundaries at episodes 350 and 600.

As shown in Figure 5b, ablation experiments are conducted by removing different state components in sequence to further verify the rationality and effectiveness of the designed state space. When relative information is removed, the learning process becomes significantly unstable, with slower convergence and lower final rewards. This indicates that relative distance is critical for inter-train coordination and safety-aware decision-making. When acceleration information is removed, the performance also degrades, although to a lesser extent, mainly affecting convergence smoothness and control stability.

Overall, the full state design achieves the best performance in terms of both convergence speed and stability. These results demonstrate that the proposed state representation is not redundant. Instead, each component contributes meaningfully to the learning process.

4.3. Performance Comparison of Different Algorithms

To verify the performance advantages of the proposed PM-MASAC algorithm in the multi-agent train virtual coupling task, several representative multi-agent reinforcement learning algorithms are selected as baseline methods, including MATD3 (Multi-Agent Twin Delayed Deep Deterministic Policy Gradient) [37], MADDPG (Multi-Agent Deep Deterministic Policy Gradient) [38], and MAPPO (Multi-Agent Proximal Policy Optimization) [39]. Among them, MATD3 and MADDPG represent multi-agent methods based on deterministic policy gradients, whereas MASAC and MAPPO adopt stochastic policy modeling with explicit exploration mechanisms. All algorithms are trained under the same environment configuration and share common hyperparameters, while the algorithm-specific hyperparameters are listed in Table 4. Figure 6 presents the cumulative reward curves of different algorithms during the training process.

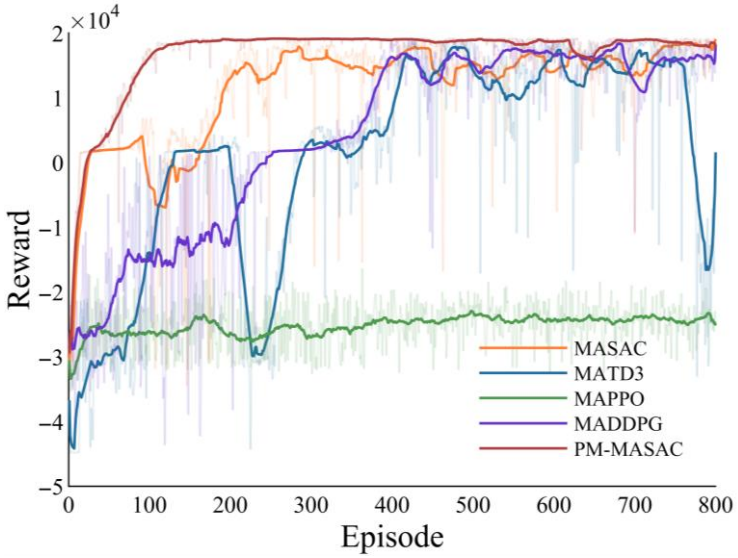


Figure 6. Training reward comparison of different algorithms.

Table 4. Specific hyperparameters of MATD3 and MAPPO.

Algorithm	Policy Delay	Target Policy Noise	Target Noise Clip
MATD3	2	0.2	0.5
Algorithm	GAE λ	Entropy coefficient	Clip Epsilon
MAPPO	0.95	0.01	0.2

It can be observed that the proposed PM-MASAC algorithm achieves a faster convergence rate in the early training stage and exhibits smaller reward fluctuations throughout the training process, eventually stabilizing at a higher reward level. Compared with deterministic policy methods, the proposed algorithm demonstrates stronger exploration capability and improved training stability in complex multi-train interaction scenarios. Moreover, relative to MAPPO and the original MASAC algorithm, PM-MASAC achieves clear advantages in both convergence speed and final reward performance, further validating the effectiveness of the proposed approach for the multi-agent train virtual coupling control task.

4.4. Ablation Study of PM-MASAC Algorithm

To further investigate the contribution of two key components in the proposed PM-MASAC framework, i.e., the Prioritized Experience Replay (PER) mechanism and the Mamba-based predictive module, ablation experiments are conducted on the PM-MASAC algorithm. Figure 7 presents the training reward curves of the ablation study of the proposed algorithm.

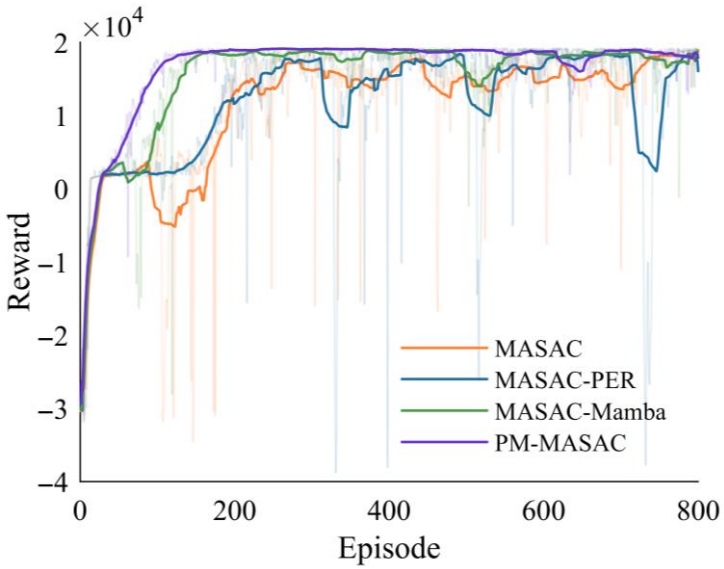


Figure 7. Reward curves of ablation study of PM-MASAC.

The experimental results indicate that, compared with the baseline MASAC algorithm, the introduction of the Mamba prediction structure leads to faster convergence, higher final reward values, and significantly reduced reward fluctuations during the training process. This improvement can be attributed to the enhanced state representation generated by the prediction module. By incorporating predicted future state information into the critic input, the algorithm is able to capture the temporal evolution characteristics of multi-train states more effectively, thereby improving the stability of multi-agent cooperative decision-making.

Overall, the ablation results show that the PER mechanism mainly improves the training efficiency by prioritizing informative experiences, while the Mamba prediction

module plays a key role in enhancing policy stability and final performance. These two components complement each other within the PM-MASAC framework, enabling the proposed method to achieve faster convergence and superior overall control performance in the multi-agent train virtual coupling task.

In addition, to analyze the influence of the predictive model structure, the proposed Mamba-based prediction module is compared with the conventional LSTM prediction model and MLP model. Figure 8 presents the training reward curves of different prediction models.

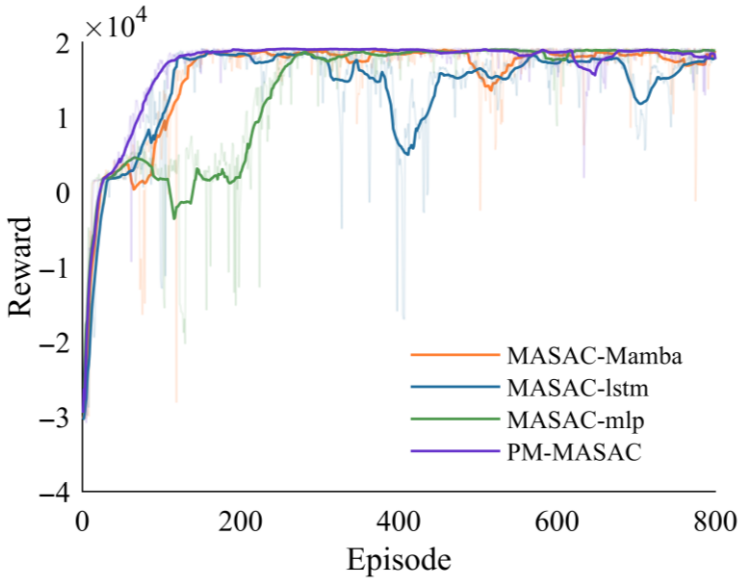


Figure 8. Reward curves of different prediction modules.

In comparison with the LSTM-based prediction model, the reward curve of the Mamba-based model is smoother throughout the entire training stage, whereas the LSTM-based model shows obvious and frequent oscillations. Moreover, compared with the MLP-based prediction model, the Mamba-based approach achieves faster initial convergence. The observation further confirms the advantages of Mamba in complex multi-agent temporal modeling tasks.

4.5. Impact of Observation Delay on System Performance

In real-world railway operations, information exchange among trains is constrained by communication conditions, where state information cannot always be obtained instantaneously. Such non-ideal sensing and information conditions introduce delayed observation feedback, which may affect the decision-making accuracy and control performance of multi-train cooperative strategies.

Therefore, incorporating observation delays into the simulation is essential for evaluating the applicability and robustness of the proposed method under realistic operating conditions. In this study, a fixed observation delay of 3 simulation steps is introduced. Figure 9 presents the reward curve under the delayed observation setting and compares it with the delay-free case. The tracking performance of algorithms under observation delay is shown in Figure 10.

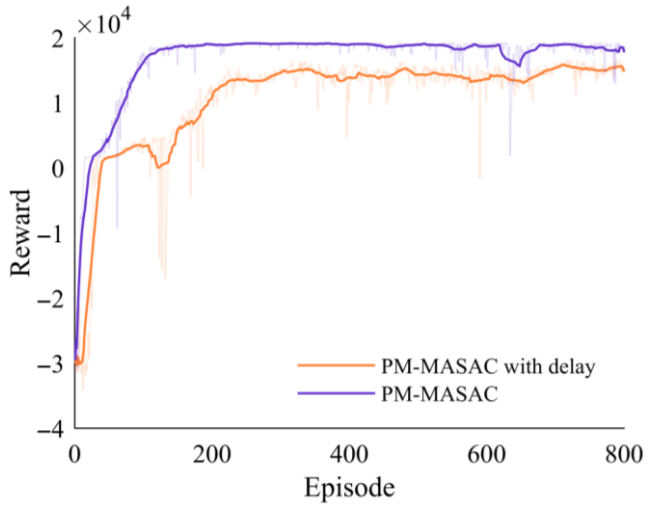


Figure 9. Performance comparison of algorithms with communication delay.

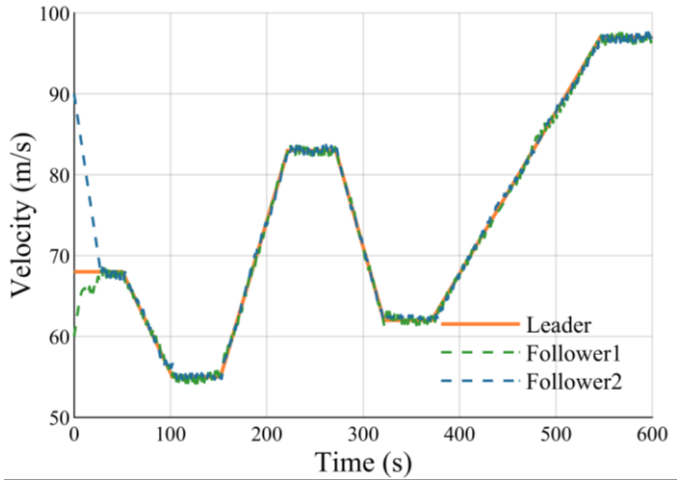


Figure 10. Training Results Curve of algorithms with communication delay.

Compared with the delay-free case, the cumulative reward exhibits a slight degradation and minor fluctuations during training. This phenomenon is mainly attributed to the delayed state feedback, which leads to a mismatch between the agents' current states and their perceived states, thereby inducing small oscillations in the reward signal. However, as reflected in the tracking results, these reward fluctuations do not significantly affect the overall control quality, since both inter-vehicle distance and velocity errors remain consistently low. The tracking data under the delayed observation setting is presented in Table 5.

Table 5. Control performance of algorithms with communication delay.

Algorithms	$t_s(s)$	$v_e(\%)$	$d_e(\%)$	MSE_v (m^2/s^2)	**Compliance Rate (%)**
PM-MASAC with delay	24.9	0.52	2.5	4.1476	93.45

Under this setting, the proposed method maintains strong formation control performance. Quantitatively, the average velocity error is only 0.52%, and the formation success rate reaches 93.45%. The results indicate that although observation delay increases the difficulty of coordination and slightly reduces the reward magnitude, the proposed approach still achieves stable and accurate platoon formation with strong robustness.

4.6. Training Scenario Result Analysis

In the training scenario, the train speed profile is designed to simulate realistic operating conditions of high-speed railways, allowing the proposed virtual coupling control strategy to be evaluated under non-stationary conditions. The train operating states and action constraints are listed in Table 6.

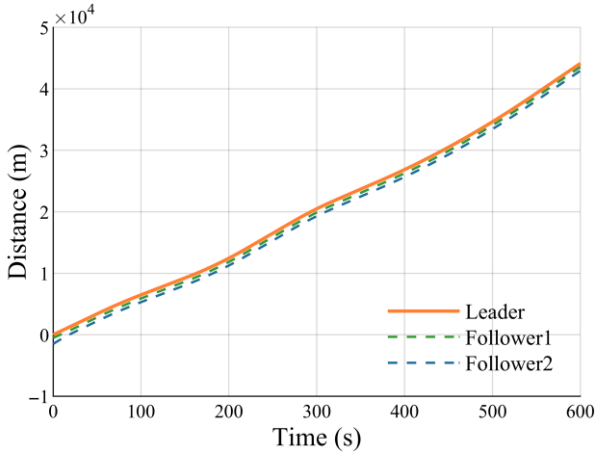
Table 6. Constraints of state and action.

Meaning	Value
Maximum speed (m/s)	105
Acceleration range (m/s^2)	$(-0.8, 0.8)$
Minimum safety interval (m)	150
Ideal tracking interval (m)	600

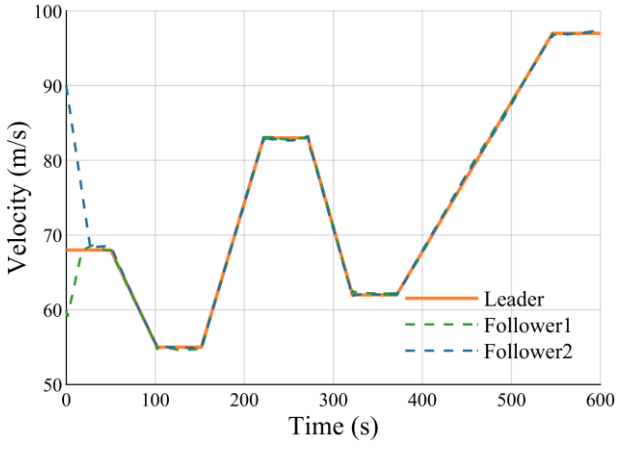
At the initial stage of training, the leader operates in a steady cruising status, while two following trains start with different initial velocities and are positioned 500 m and 1500 m behind the leader, respectively. This setup forms an initial operating condition containing both speed and spacing disturbances.

The velocity profile of the leading train is designed as a multi-stage operating scenario containing two temporary speed adjustment processes. In the first stage, the train decelerates to a lower cruising speed and then accelerates to recover its operating speed. A second deceleration–acceleration cycle with different acceleration and deceleration rates is further introduced to represent another operational disturbance. After the disturbance is cleared, the train gradually accelerates and finally reaches a stable high-speed cruising state.

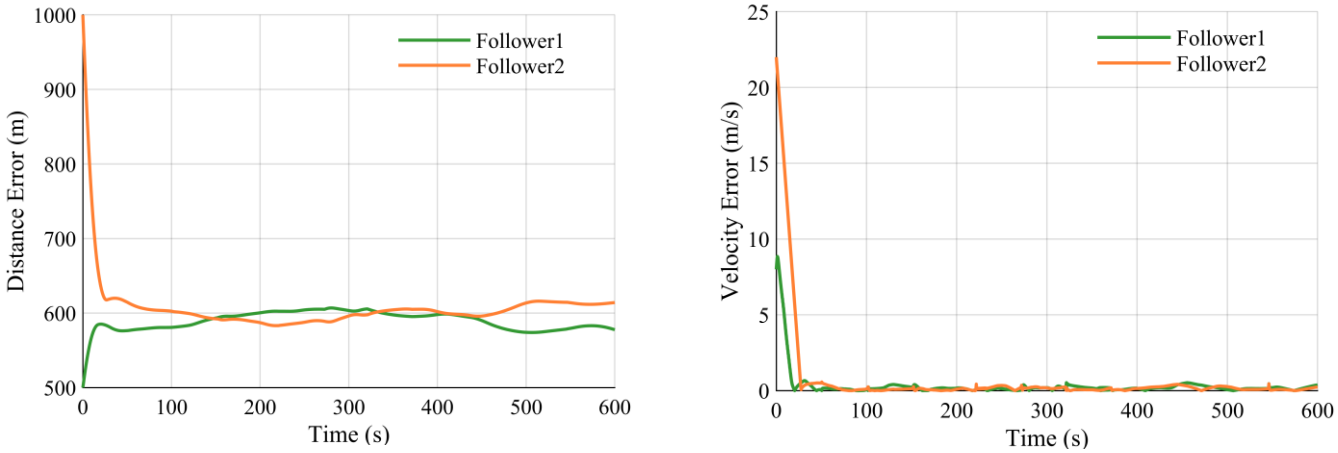
The above speed profile covers several typical operating conditions in high-speed railway systems, including cruising, speed-restricted operation, acceleration recovery, and dispatch intervention. Figure 11 illustrates the train speed curves, displacement curves, and the corresponding speed and spacing errors during the tracking process.



(**a**)



(**b**)



(**c**) (**d**)

Figure 11. Training Results Curve: (**a**) position trajectory; (**b**) speed trajectory; (**c**) spacing error; (**d**) speed error.

As shown in Figure 11, all trains achieve a stable virtual coupling state and maintain consistent position trajectories during operation. The speed of the followers tracks the speed of the leader with rapid response and negligible fluctuation. The spacing error remains within a narrow range around the desired distance, while the speed error rapidly converges to a near-zero level and remains stable during the operation phase.

To further quantitatively evaluate the effectiveness of the proposed method, comprehensive comparisons with several algorithms are conducted in Table 7. The results show that PM-MASAC achieves the best overall control performance among the compared methods. In particular, the proposed algorithm attains a virtual coupling compliance rate of 95.7%, indicating that the trains remain within the desired spacing and speed error bounds for the majority of the operation period.

Combined with the above qualitative and quantitative analysis, it can be concluded that the proposed method not only ensures stable and accurate tracking control but also possesses stronger overall performance and robustness compared with other algorithms.

Table 7. Control performance of different algorithms in the training scenario.

Algorithms	$t_s(s)$	$\mathbf{v}_e(\%)$	$\mathbf{d}_e(\%)$	$\mathbf{MSE}_v$ $\left(\mathbf{m}^2/\mathbf{s}^2\right)$	**Compliance Rate (%)**
MADDPG	74.55	0.46	2.12	5.9735	78.26
MATD3	20.4	0.49	2.4	4.5196	85.81
MASAC	46.65	0.5	1.77	4.0862	86.96
PM-MASAC	20.7	0.5	1.48	4.2575	95.7

4.7. Generalization Performance Analysis

To further evaluate the generalization capability of the proposed algorithm, we evaluate the proposed algorithm on four distinct and representative operational scenarios that were never encountered during training: cruising operation (CR), speed regulation (SR), station departure and arrival (DA), and real-route speed profile simulation (RRS). These test scenarios are designed with different speed profiles, disturbance patterns, and dynamic characteristics from the training environment. A robust policy should extract generalizable control principles rather than merely memorize training conditions. Therefore,

successful performance in these unseen conditions provides strong evidence of the policy's robustness against operational uncertainties.

1. Case 1: Cruising Condition (CR)

In this scenario, the initial velocities of follower 1 and follow 2 are set to 65 m/s and 88 m/s, respectively, with initial positions located 400 m and 1100 m behind the leader. The reference speed profile of the leader covers typical operating speeds of high-speed railways, including 245 km/h, 300 km/h, and 350 km/h. Figure 12 presents the corresponding velocity tracking curves of the trains under these cruising conditions.

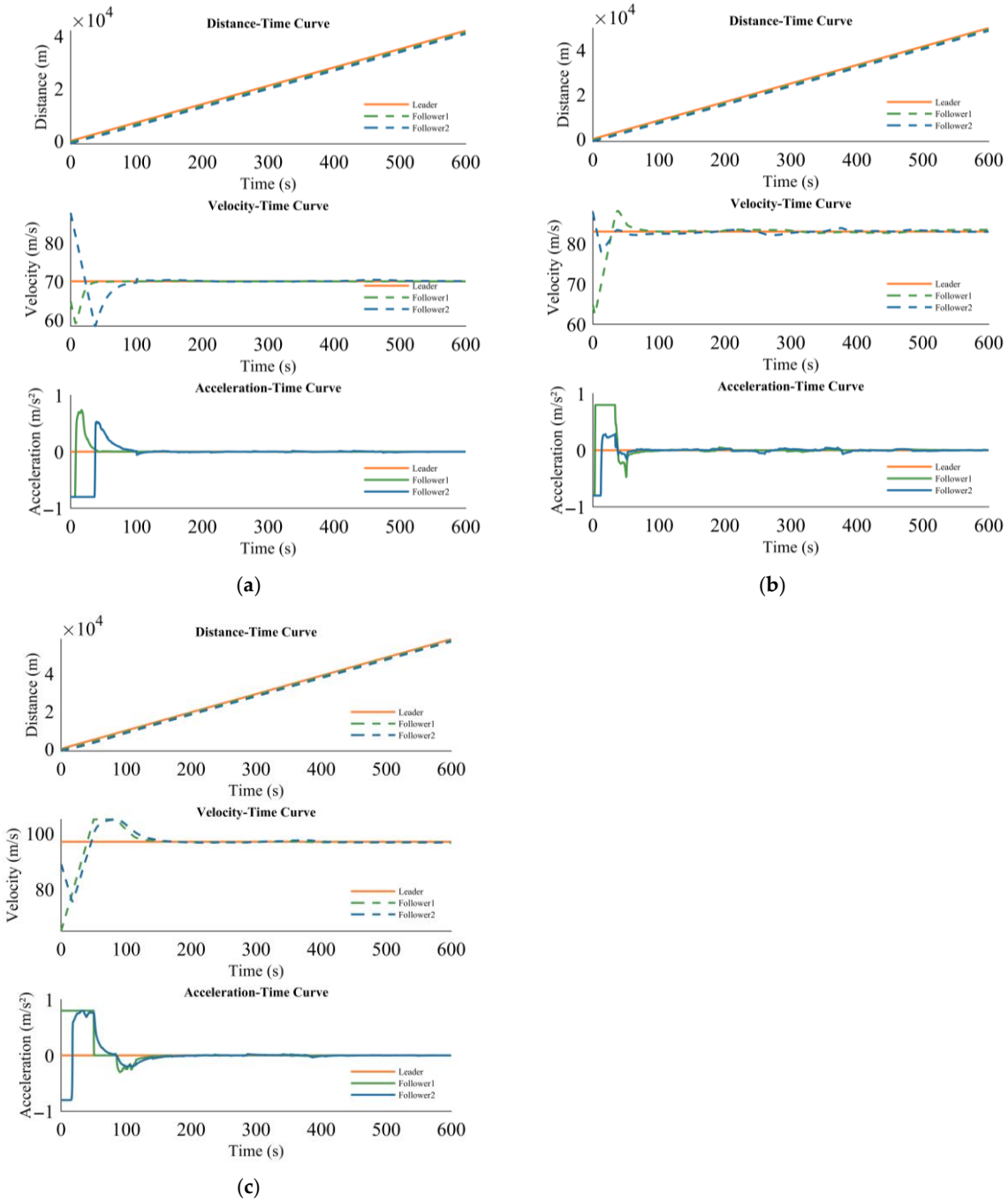


Figure 12. Train tracking trajectories of the cruising conditions: (**a**) 70 m/s; (**b**) 83 m/s; (**c**) 97 m/s.

For quantitative analysis of the control performance of different algorithms during the steady cruising stage, a cruising speed of 70 m/s is selected for performance evalua-

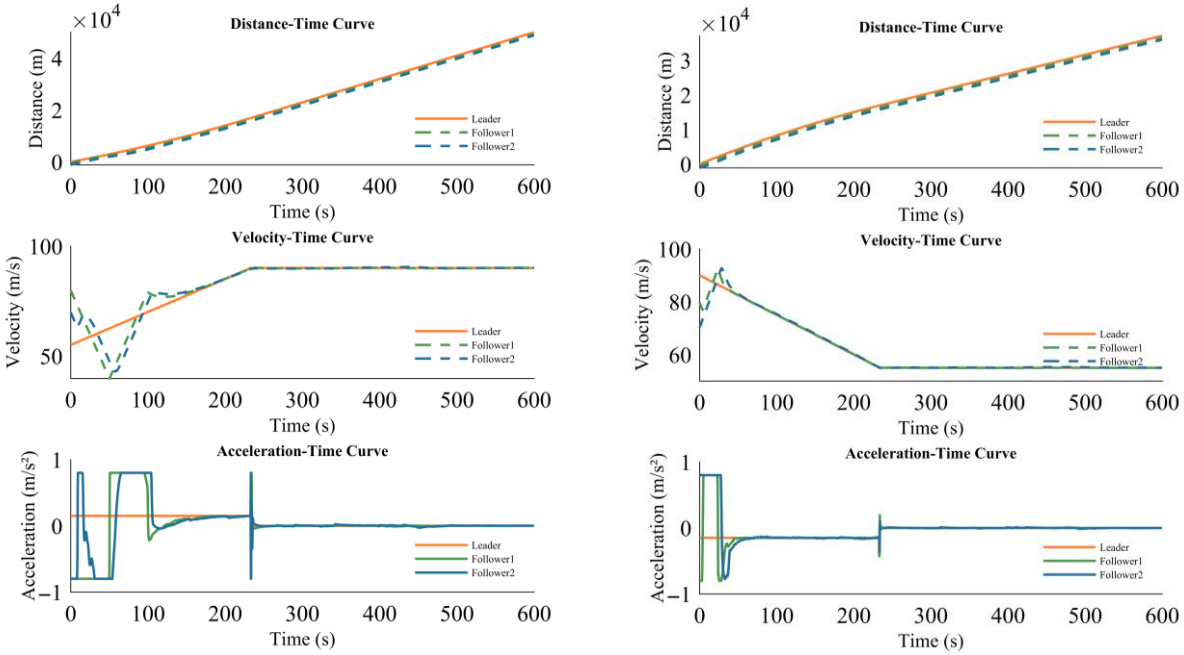
tion. The control performance of different algorithms is summarized in Table 8. The results show that all algorithms are capable of tracking the motion of the leader and maintaining the desired inter-train spacing during operation. However, compared with other algorithms, the proposed PM-MASAC algorithm achieves the highest virtual coupling of 74.25%, outperforming MASAC (64.65%) and MATD3 (54.3%). In addition, the acceleration error is reduced to 0.9963, which is lower than that of the baseline algorithms, indicating improved control smoothness. compliance rate and better ride comfort. The results demonstrate the PM-MASAC algorithm achieve superior overall control performance under cruising conditions.

Table 8. Control performance (70 m/s) of different algorithms in case CR.

Algorithms	$t_s(s)$	$v_e(\%)$	$d_e(\%)$	MSE_v (m^2/s^2)	MSE_a (m^2/s^4)	Compliance Rate (%)
MADDPG	84.45	0.19	8.31	21.8764	1.5953	37.3
MATD3	80.55	0.32	4.36	5.4507	1.4320	54.3
MASAC	65.55	0.35	5.34	9.185	1.2431	64.65
PM-MASAC	48	0.32	2.92	5.6086	0.9963	74.25

2. Case 2: Speed Regulation (SR)

In this scenario, the initial velocities of follower 1 and follower 2 are 80 m/s and 70 m/s, respectively, with initial positions 500 m and 1100 m behind the leader. Two acceleration magnitudes (0.15 and 0.4 m/s²) are applied to the leader for both acceleration and deceleration to simulate speed adjustments caused by dispatching commands or operational constraints during inter-station operation. The corresponding train tracking trajectories are illustrated in Figure 13, while the quantitative performance comparison of different algorithms is summarized in Table 9.



(**a**) (**b**)

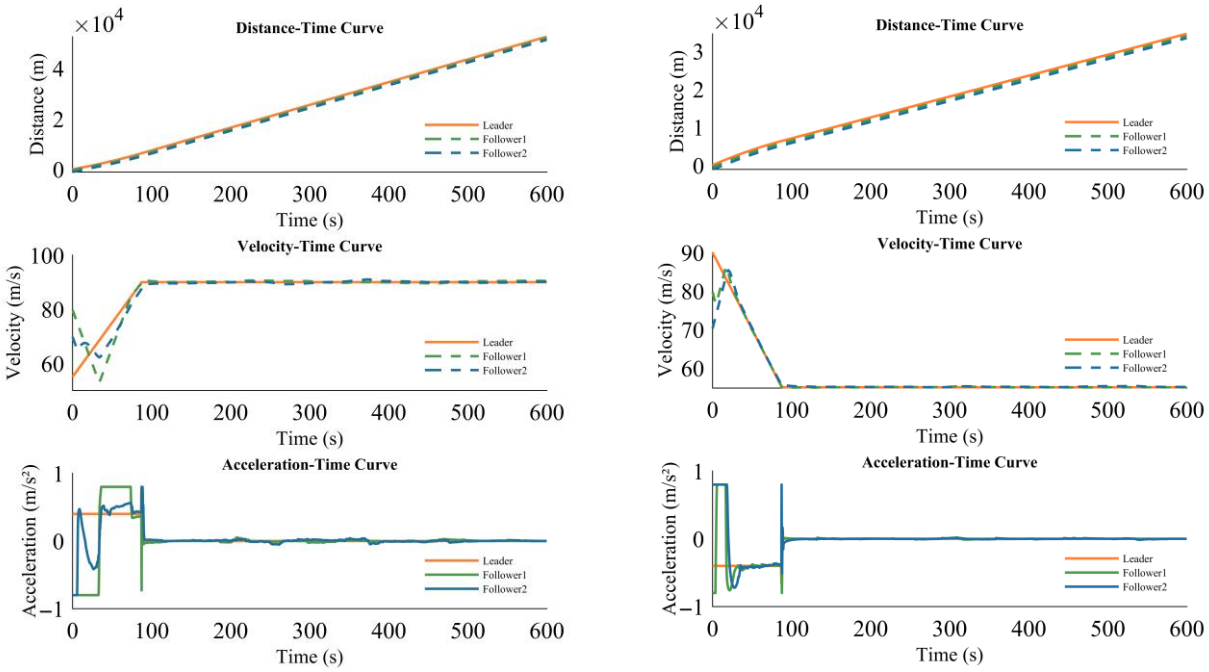


Figure 13. Tracking trajectories of different acceleration magnitudes: (**a**) 0.15m/s^2; (**b**) -0.15m/s^2; (**c**) 0.4m/s^2; (**d**) -0.4m/s^2.

Table 9. Control performance (0.15m/s^2) of different algorithms in Case SR.

Algorithms	$t_s(s)$	$\mathbf{v_e}(\%)$	$\mathbf{d_e}(\%)$	$\mathbf{MSE_v}$ $(\mathbf{m^2/s^2})$	$\mathbf{MSE_a}$ $(\mathbf{m^2/s^4})$	**Compliance Rate (%)**
MADDPG	278.25	0.05	12.38	6.6043	2.749	10.90
MATD3	163.5	0.11	4.71	10.5468	2.8098	57.20
MASAC	98.85	0.24	4.21	11.8012	2.3832	60.9
PM-MASAC	88.65	0.23	2.76	25.705	0.1046	74.40

The results indicate that the proposed PM-MASAC algorithm can effectively suppress both velocity and spacing deviations under different acceleration and deceleration scenarios. The trains reach a stable virtual coupling state within a relatively short adjustment period of 88.65 s, which outperforms the comparative algorithms. Moreover, the algorithm maintains smooth acceleration profiles, ensuring passenger comfort while achieving a higher virtual coupling success rate of 74.4% than the other algorithms.

3. Case 3: Departure and Arrival (DA)

This scenario simulates the station departure and arrival process. The initial conditions approximately achieve the virtual coupling state. The initial velocities of follower 1 and follower 2 are 40 m/s and 50 m/s, respectively, with initial positions 600 m and 1200 m behind the leader.

During operation, the leading train decelerates from 55 m/s to a stop at a deceleration of 0.2 m/s^2, remains stationary at the station for 20 s, and then accelerates again with the same acceleration magnitude to depart from the station. The corresponding tracking curves are shown in Figure 14, while the quantitative performance comparison is summarized in Table 10. "–" denotes that the method does not satisfy the steady-state condition and thus the compliance rate and t_s cannot be recorded.

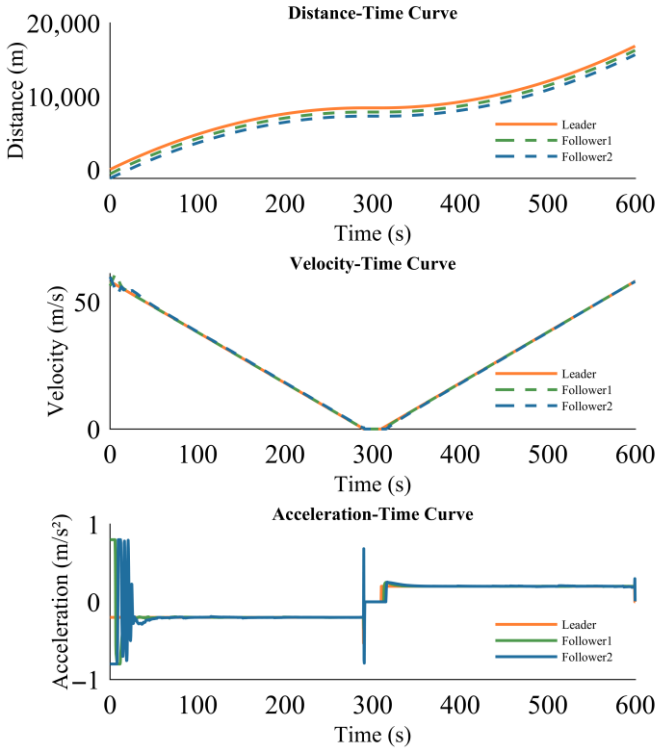


Figure 14. Train tracking trajectories of Case DA.

Table 10. Control performance of different algorithms in Case DA.

Algorithms	$t_s(s)$	$\mathbf{v_e}(\%)$	$\mathbf{d_e}(\%)$	$\mathbf{MSE_v}$ $\left(\mathbf{m^2/s^2}\right)$	$\mathbf{MSE_a}$ $\left(\mathbf{m^2/s^4}\right)$	**Compliance Rate (%)**
MADDPG	-	3.84	18.97	1.739	0.427	-
MATD3	-	11.36	15.78	9.5509	0.2309	-
MASAC	1.65	19.95	23.11	58.6123	0.4364	0.10
PM-MASAC	1.05	0.83	3	0.1618	0.0161	71.15

The Departure and Arrival (DA) scenario introduces a fundamentally different dynamic structure. The leading train undergoes a complete motion regime transition, starting from a deceleration phase, followed by a zero-speed stop phase, and subsequently a re-acceleration phase for departure. This introduces a temporary loss of dynamic continuity during the stop phase, as well as abrupt changes in the relative motion structure between trains when switching between stationary and moving states.

Due to the significant dynamic changes, several baseline algorithms fail to maintain a stable virtual coupling state throughout the entire operation. In contrast, the proposed PM-MASAC achieves a compliance rate of 71.15%, demonstrating its strong robustness under complex operating conditions. In addition, the $\mathrm{MSE_v}$ and MSE_a errors are significantly reduced to 0.1618 and 0.0161, respectively. The result demonstrates the superior stability and adaptability of the proposed control strategy.

4. Case 4: Real Route Simulation

To further evaluate the applicability of the proposed method in realistic railway environments, a comprehensive operating scenario is constructed to simulate multiple typical operational states and transitions. The overall speed and acceleration settings comply with engineering constraints of high-speed railway operations. The corresponding train

tracking trajectories are presented in Figure 15, and the quantitative performance comparison of different algorithms is summarized in Table 11.

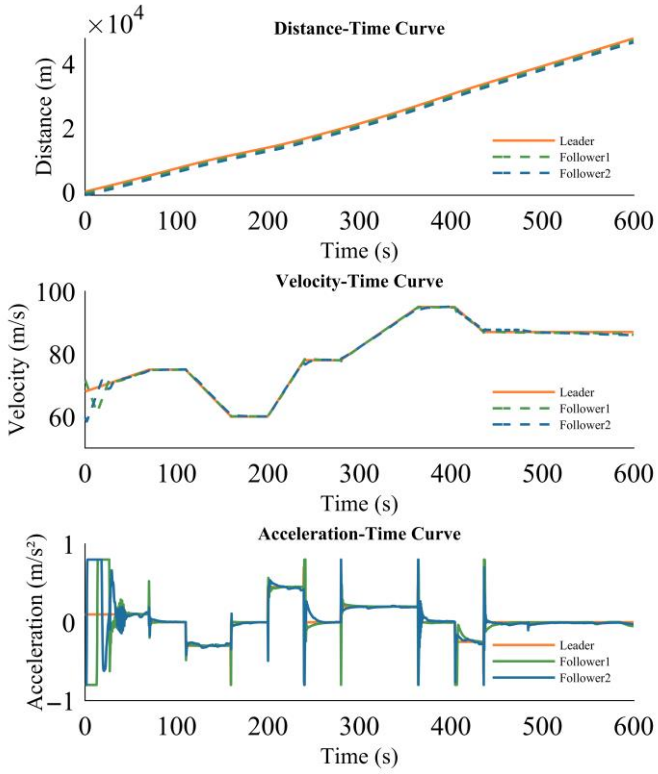


Figure 15. Train tracking trajectories of Case SSR.

Table 11. Control performance of different algorithms in Case SSR.

Algorithms	$t_s(s)$	$\mathbf{v_e}(\%)$	$\mathbf{d_e}(\%)$	$\mathbf{MSE_v}$ $\left(\mathbf{m^2/s^2}\right)$	$\mathbf{MSE_a}$ $\left(\mathbf{m^2/s^4}\right)$	**Compliance Rate (%)**
MADDPG	18.45	0.16	2.36	1.7306	0.6895	37.25
MATD3	26.1	0.11	2.89	2.9514	0.6118	38.15
MASAC	25.95	0.39	1.76	1.5162	0.3	77.60
PM-MASAC	21.15	0.29	1.8	1.0296	0.2762	94.25

As shown in Table 11, the proposed PM-MASAC algorithm achieves a virtual coupling compliance rate of 94.25%, which is considerably higher than that of MASAC and other baseline algorithms. Meanwhile, the velocity and acceleration errors are maintained at low levels, with the MSE_v and MSE_a reduced to 1.0296 and 0.2762, respectively. These results confirm that PM-MASAC is capable of ensuring stable tracking performance and smooth control behavior even under complex operating conditions involving multiple speed transitions. This further demonstrates the strong robustness and generalization capability of the proposed method.

In a word, although the dynamic responses in each case exhibit different characteristics, the proposed PM-MASAC framework consistently demonstrates stable tracking behavior and cooperative control capability. To clearly compare the overall performance of the algorithms, the results of the four cases are summarized using three key indicators: the distance tracking error, the mean squared error of acceleration (MSEA), and the virtual coupling

compliance rate. These indicators comprehensively reflect the tracking accuracy, operational smoothness, and coupling stability of the virtual coupling system. The statistical results of these indicators across Case 1–4 are summarized in Figure 16.

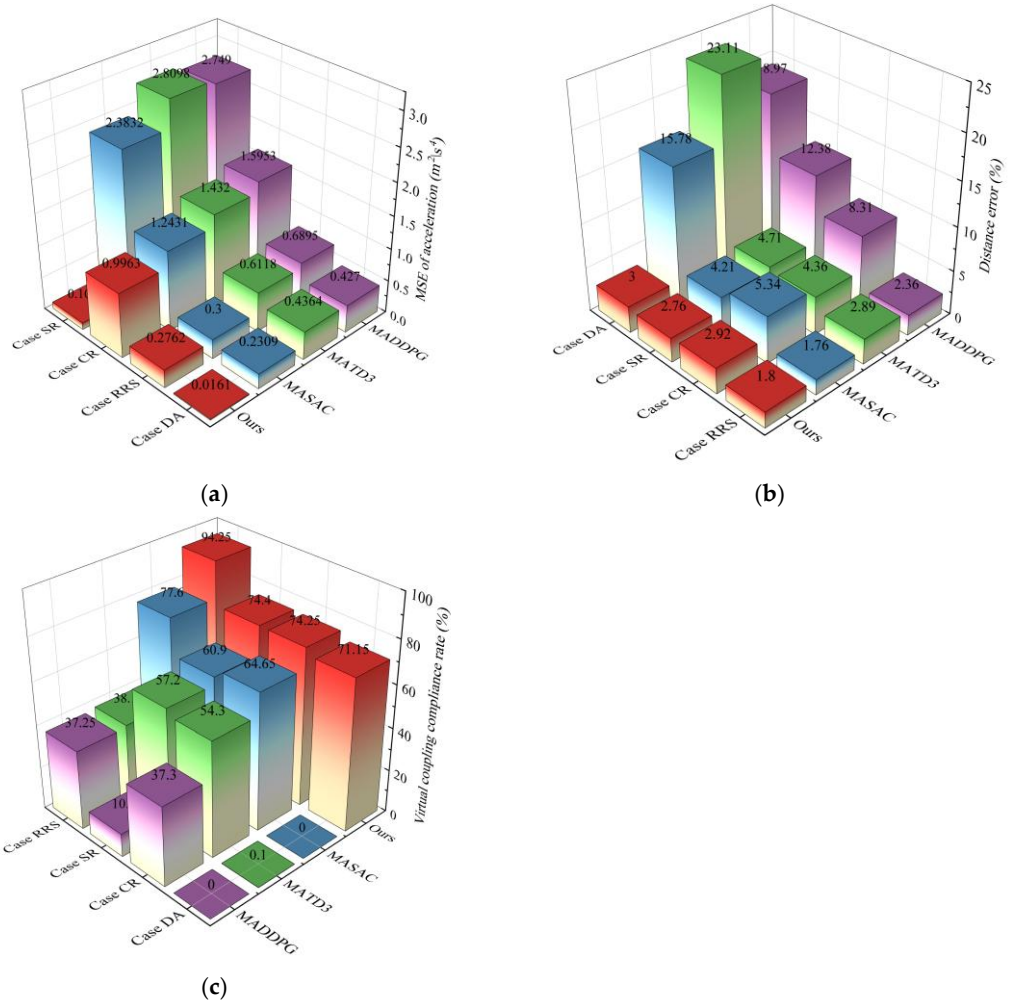


Figure 16. Performance comparison of different algorithms: (**a**) MSE_a; (**b**) distance error d_e; (**c**) virtual coupling compliance rate.

As illustrated in the bar charts, MADDPG and MATD3 exhibit relatively large distance errors and noticeable acceleration fluctuations in most cases, resulting in lower steady-state compliance rates of the virtual coupling formation. Their control stability becomes particularly insufficient under scenarios with stronger dynamic variations.

After introducing the entropy-regularized exploration mechanism, MASAC shows certain performance improvements in some scenarios; however, its performance still fluctuates noticeably with changes in operating conditions.

In contrast, PM-MASAC demonstrates the most stable overall performance across all test scenarios. It maintains lower distance and acceleration mean squared errors while achieving the highest steady-state compliance rate. The advantage becomes particularly evident under non-stationary conditions, indicating that the proposed method possesses stronger robustness and generalization capability for the virtual coupling task.

https://doi.org/10.3390/electronics15091823

5. Conclusions

To address the challenges of complex multi-train interactions and significant long-term temporal dependencies in train virtual coupling control, this paper proposes a Predictive Mamba-based Multi-Agent Soft Actor-Critic (PM-MASAC) control framework under the Centralized Training and Decentralized Execution (CTDE) architecture. By introducing the Mamba prediction model to construct enhanced state representations for the centralized Critic network, significantly improving the modeling capacity of the system. Integrated with a prioritized experience replay mechanism and an automatic entropy adjustment strategy, the proposed method enhances sample efficiency and exploration stability within continuous action spaces. Furthermore, a hierarchical reward function for multi-agent virtual coupling tasks is designed to guide agents in learning coordinated car-following control strategies.

Simulation experiments were conducted across various non-stationary simulation scenarios consistent with high-speed railway operation characteristics. Compared to typical multi-agent reinforcement learning algorithms such as MASAC, MADDPG, and MATD3, the proposed PM-MASAC framework achieves more stable speed and distance tracking control under complex operating conditions. Moreover, it achieves superior comprehensive performance in key indicators such as tracking accuracy, ride comfort, and steady-state compliance of the virtual coupling distance. These results verify the effectiveness of the introduced Mamba-based state prediction mechanism in enhancing the stability and robustness of multi-agent cooperative control.

In summary, this paper presents a cooperative control framework that combines predictive models with multi-agent reinforcement learning for the train virtual coupling control problem from the perspectives of system state evolution modeling, improving control stability in complex collaborative scenarios.

In terms of simulation environment modeling for data-driven training, future research could be extended in two directions. First, adopting a multi-mass model that captures inertial effects between carriages to enable finer motion control. Second, moving beyond simulation-based modeling to directly leverage real train operational data for training agents and learning control policies. In addition, dynamically increasing inter-train spacing and incorporating routing-based decoupling strategies near stations represent important and practical extensions of the current framework, which can improve operational flexibility and safety in real-world deployment scenarios. On the algorithmic side, future work will further investigate longer prediction horizons and multi-scale state modeling methods, while validating the framework's generalization capability and engineering applicability under large-scale coupling and complex uncertain disturbances.

Author Contributions: Conceptualization, H.H. and Q.F.; methodology, H.H. and Q.F.; software, H.H. and Z.H.; validation, H.H., Z.H. and W.L.; formal analysis, H.L.; investigation, Q.F. and W.L.; resources, H.L.; data curation, Z.H. and W.L.; writing—original draft preparation, H.H.; writing—review and editing, H.H. and Q.F.; visualization, H.H.; supervision, Q.F.; project administration, Q.F.; funding acquisition, H.L. All authors have read and agreed to the published version of the manuscript.

Funding: This research was funded by Research on Intelligent Health Management for Railway Switch Machines Based on Full Lifecycle Multi-Source Heterogeneous Big Data, LJ242510150003.

Data Availability Statement: Data are contained within the article.

Conflicts of Interest: The authors declare that the research was conducted in the absence of any commercial or financial relationships that could be construed as a potential conflict of interest.

Abbreviations

The following abbreviations are used in this manuscript:

VBTC	Vehicle-to-Vehicle Communication Based Train Control
VC	Virtual coupling
MPC	Module predictive control
PID	Proportional integral derivative
PER	Prioritized experience replay
CFD	Computational Fluid Dynamics
ILQR	Iterative Linear Quadratic Regulator
DQN	Deep Q-Network
DDPG	Deep Deterministic Policy Gradient
UAV	Unmanned Aerial Vehicles
LSTM	Long Short-Term Memory
MASAC	Multi-Agent Soft Actor-Critic
RMSNorm	Root Mean Square Layer Normalization
SILU	Sigmoid Linear Unit
Hippo	High-Order Polynomial Projection Operators
MATD3	High-Order Polynomial Projection Operators
MAPPO	Multi-Agent Proximal Policy Optimization
MADDPG	Multi-Agent Deep Deterministic Policy Gradient

References

1. Gao, Y.B.; Yan, L.X.; Li, M.Q.; Tian, Z.Y. Research on movement authorization generation of VBTC based on timed automata. In Proceedings of the Advances in Natural Computation, Fuzzy Systems and Knowledge Discovery (ICNC-FSKD 2020), Xi'an, China, 1–3 August 2020.
2. Luo, C.; Liu, Y.; Li, Y.; Sun, Z.; Hu, X.; Liu, J. Dynamic decoupling control for virtually coupled train set: A reinforcement learning-based model predictive control approach. In Proceedings of the IEEE 27th International Conference on Intelligent Transportation Systems (ITSC), Edmonton, AB, Canada, 24–27 September 2024. https://doi.org/10.1109/ITSC58415.2024.10920128.
3. Schumann, T. Increase of capacity on the Shinkansen high-speed line using virtual coupling. *Int. J. Transp. Dev. Integr*. **2017**, *1*, 666–676. https://doi.org/10.2495/tdi-v1-n4-666-676.
4. Tian, S.; Yan, F.; Shang, W.-L.; Majumdar, A.; Chen, H.; Chen, M.; Zeinab, M.; Tian, Y. Energy consumption analysis of trains based on multi-mode virtual coupling operation control strategies. *Appl. Energy*. **2025**, *377*, 124684. https://doi.org/10.1016/j.apenergy.2024.124684.
5. Fadhel, S.; Capgemini, S.F. Quantification of the aerodynamic gain on two virtually coupled modular trains. *Int. J. Comput. Methods Exp. Meas*. **2022**, *10*, 372–384. https://doi.org/10.2495/CMEM-V10-N4-372-384.
6. Knospe, C. PID control. *IEEE Control. Syst. Mag*. **2006**, *26*, 30–31. https://doi.org/10.1109/MCS.2006.1580151.
7. Kalman, R.E. Contributions to the theory of optimal control. *Bol. Soc. Mat. Mex*. **1960**, *5*, 102–119.
8. Darby, M.L.; Nikolaou, M. MPC: Current practice and challenges. *Control Eng. Pract*. **2012**, *20*, 328–342.
9. Shtessel, Y.; Edwards, C.; Fridman, L.; Levant, A. *Sliding Mode Control and Observation*; Birkhäuser: New York, NY, USA, 2014.
10. Liu, Y.; Zhou, Y.; Su, S.; Xun, J.; Tang, T. Control strategy for stable formation of high-speed virtually coupled trains with disturbances and delays. *Comput.-Aided Civ. Infrastruct. Eng*. **2023**, *38*, 621–639. https://doi.org/10.1111/mice.12873.
11. He, R.; Zhang, Y. An adaptive sliding mode controller for virtual coupling high-speed train. In Proceedings of the Chinese Control Conference (CCC), Tianjin, China, 24–26 July 2023. https://doi.org/10.23919/CCC58697.2023.10240517.
12. Luo, X.; Gao, J.; Zhang, Y.; Han, X.; Liu, H. A feedforward PID control approach for reference tracking of virtually coupled train set. In Proceedings of the IEEE 26th International Conference on Intelligent Transportation Systems (ITSC), Bilbao, Spain, 24–28 September 2023. https://doi.org/10.1109/ITSC57777.2023.10422350.
13. Du, H.; Hu, W.; Liu, R.; Min, S. ILQR-based robust control approach for virtual coupled train set. In Proceedings of the International Conference on Neural Networks, Information and Communication Engineering (NNICE), Guangzhou, China, 10–12 January 2025. https://doi.org/10.1109/NNICE64954.2025.11064517.
14. Felez, J.; Vaquero-Serrano, M.A.; de Dios Sanz, J. A Robust Model Predictive Control for Virtual Coupling in Train Sets. *Actuators* **2022**, *11*, 372. https://doi.org/10.3390/act11120372.

15. Xie, Y.; Yang, H. A review of artificial intelligence in train driving and control. *Railw. Sci.* **2025**, *4*, 762–782. https://doi.org/10.1108/RS-09-2025-0036.
16. Kaelbling, L.P.; Littman, M.L.; Moore, A.W. Reinforcement learning: A survey. *J. Artif. Intell. Res.* **1996**, *4*, 237–285.
17. Dun, Y.; ShangGuan, W.; Song, H.; Cai, B. Time-space-based virtual coupling high-speed train separation model and trajectory planning. *IEEE Trans. Intell. Transp. Syst*. **2024**, *25*, 18573–18590. https://doi.org/10.1109/TITS.2024.3442212.
18. Shang, M.; Zhou, Y.; Mei, Y.; Zhao, J.; Fujita, H. Energy-saving train operation synergy based on multi-agent deep reinforcement learning on spark cloud. *IEEE Trans. Veh. Technol*. **2023**, *72*, 214–226. https://doi.org/10.1109/TVT.2022.3205379.
19. Ben Elallid, B.; Benamar, N.; Senhaji Hafid, A.; Rachidi, T.; Mrani, N. A comprehensive survey on the application of deep and reinforcement learning approaches in autonomous driving. *J. King Saud Univ. Comput. Inf. Sci.* **2022**, *34*, 7366–7390. https://doi.org/10.1016/j.jksuci.2022.03.013.
20. Yue, P.; Jin, Y.; Dai, X.; Feng, Z.; Cui, D. Reinforcement learning for online dispatching policy in real-time train timetable re-scheduling. *IEEE Trans. Intell. Transp. Syst.* **2024**, *25*, 478–490. https://doi.org/10.1109/TITS.2023.3305074.
21. Lang, Y.; Liu, H.; Luo, X.; Lin, B. DQN-based speed curve optimization for virtual coupling. In Proceedings of the IEEE 25th International Conference on Intelligent Transportation Systems (ITSC), Macau, China, 8–12 October 2022. https://doi.org/10.1109/ITSC55140.2022.9922169.
22. Basile, G.; Lui, D.G.; Petrillo, A.; Santini, S. Deep deterministic policy gradient-based virtual coupling control for high-speed train convoys. In Proceedings of the IEEE International Conference on Networking, Sensing and Control (ICNSC), Shanghai, China, 15–18 December 2022. https://doi.org/10.1109/ICNSC55942.2022.10004067.
23. Chen, B.; Zhang, L.; Cheng, G.; Liu, Y.; Chen, J. A novel approach for train tracking in virtual coupling based on soft actor-critic. *Actuators* **2023**, *12*, 447. https://doi.org/10.3390/act12120447.
24. Liao, J.; Yang, G.; Zhang, S.; Zhang, F.; Gong, C. A deep reinforcement learning approach for the energy-aimed train timetable rescheduling problem under disturbances. *IEEE Trans. Transp. Electr.* **2021**, *7*, 3096–3109. https://doi.org/10.1109/TTE.2021.3075462.
25. Mao, X.; Wu, G.; Fan, M.; Cao, Z.; Pedrycz, W. DL-DRL: A double-level deep reinforcement learning approach for large-scale task scheduling of multi-UAV. *IEEE Trans. Autom. Sci. Eng*. **2025**, *22*, 1028–1044. https://doi.org/10.1109/TASE.2024.3358894.
26. Karimzadeh, M.; Aebi, R.; Souza, A.M.; Zhao, Z.; Braun, T.; Sargento, S. Reinforcement learning-designed LSTM for trajectory and traffic flow prediction. In Proceedings of the IEEE Wireless Communications and Networking Conference (WCNC), Nanjing, China, 29 March–1 April 2021. https://doi.org/10.1109/WCNC49053.2021.9417511.
27. Tseng, Y.T.; Ferng, H.W. Adaptive DRL-based traffic signal control with an infused LSTM prediction model. In Proceedings of the Advances and Trends in Artificial Intelligence (IEA/AIE 2025), Kytakyushu, Japan, 1 July 2025. https://doi.org/10.1007/978-981-96-8892-0_25.
28. Liu, M.; Zhu, Y.; Chen, Y.; Zhao, D. Enhancing reinforcement learning via transformer-based state predictive representations. *IEEE Trans. Artif. Intell.* **2024**, *5*, 4364–4375. https://doi.org/10.1109/TAI.2024.3379969.
29. Liu, H.; Huang, Z.; Mo, X.; Lv, C. Augmenting reinforcement learning with transformer-based scene representation learning for decision-making of autonomous driving. *IEEE Trans. Intell. Veh.* **2024**, *9*, 4405–4421. https://doi.org/10.1109/TIV.2024.3372625.
30. Wang, Y.; Tao, X. LocoMamba: Vision-driven locomotion via end-to-end deep reinforcement learning with Mamba. *Adv. Eng. Inform.* **2026**, *70*, 104230. https://doi.org/10.1016/j.aei.2025.104230.
31. Mustafa, S.M.; Usmani, Z.A.; Rizvi, O.; Memon, A.B.; Movania, M.M. Context aware Mamba-based reinforcement learning for social robot navigation. In Proceedings of the International Conference on Control, Mechatronics and Automation (ICCMA), London, UK, 11–13 November 2024. https://doi.org/10.1109/ICCMA63715.2024.10843924.
32. Ryu, H.; Sohn, C.B.; Kim, D.Y. Latent Mamba-DQN: Improving temporal dependency modeling in deep Q-learning via selective state summarization. *Appl. Sci*. **2025**, *15*, 8956. https://doi.org/10.3390/app15168956.
33. Schaul, T.; Quan, J.; Antonoglou, I.; Silver, D. Prioritized experience replay. *arXiv* **2015**, arXiv:1511.05952.
34. Somvanshi, S.; Islam, M.M.; Mimi, M.; Polock, S.B.B.; Chhetri, G.; Das, S. From S4 to Mamba: A comprehensive survey on structured state space models. *arXiv* **2025**, arXiv:2503.18970.
35. Gu, A.; Dao, T. Mamba: Linear-time sequence modeling with selective state spaces. *arXiv* **2024**, arXiv:2312.00752.
36. Haarnoja, T.; Zhou, A.; Abbeel, P.; Levine, S. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In Proceedings of the International Conference on Machine Learning (ICML), Stockholm, Sweden, 10–15 July 2018.
37. Fujimoto, S.; Herke, H.; David, M. Addressing function approximation error in actor-critic methods. In *International Conference on Machine Learning*; PMLR: Birmingham, UK, 2018; pp. 1587–1596.

38. Lowe, R.; Wu, Y.I.; Tamar, A.; Harb, J.; Pieter Abbeel, O.; Mordatch, I. Multi-agent actor-critic for mixed cooperative-competitive environments. In *Advances in Neural Information Processing Systems*; NIPS Foundation: San Diego, CA, USA, 2017; Volume 30.
39. Yu, C.; Akash, V.; Eugene, V.; Gao, J.; Yu, W.; Alexandre, B.; Yi, W. The surprising effectiveness of ppo in cooperative multi-agent games. In *Advances in Neural Information Processing Systems*; NIPS Foundation: San Diego, CA, USA, 2022; Volume 35.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.